

FACULTY PROFILE



PARTICULARS	DETAILS	
Name	Rupesh Kumar	
Designation	Lecturer	
Date Of Birth	11/03/1990	
Branch	INFORMATION TECHNOLOGY	
AICTE Unique Id	1-4397574488	
Education Qualification	B.Tech(IT)	
Work Experience:	Teaching	9 Yrs
	Research	NIL
	Industry	NIL
	Others	NIL
Area of Specialization	INFORMATION TECHNOLOGY	
Courses taught (Please Tick)	Diploma	✓
	Post Diploma	NIL
	Under Graduate	NIL
	Post Graduate	NIL
	Post Graduate Diploma	NIL
Research guidance (No. of Students)	Master	NIL
	PhD-	NIL
No. of papers published in	National Journals	NIL
	International Journal	NIL
	Conferences	NIL
Master (Completed /Ongoing)	ONGOING	
PhD (Completed/Ongoing)	NIL	
Projects Carried Out	NIL	
Patents(Filed & Granted)	NIL	
Technology Transfer	NIL	
Research Publications (No. of papers published in National/International/Journals/Conferences)	NIL	
No. of Books published with Details(Name of Book, Publisher with ISBN, Year of Publication etc.)	NIL	

OOP Using C++

Unit 1:-

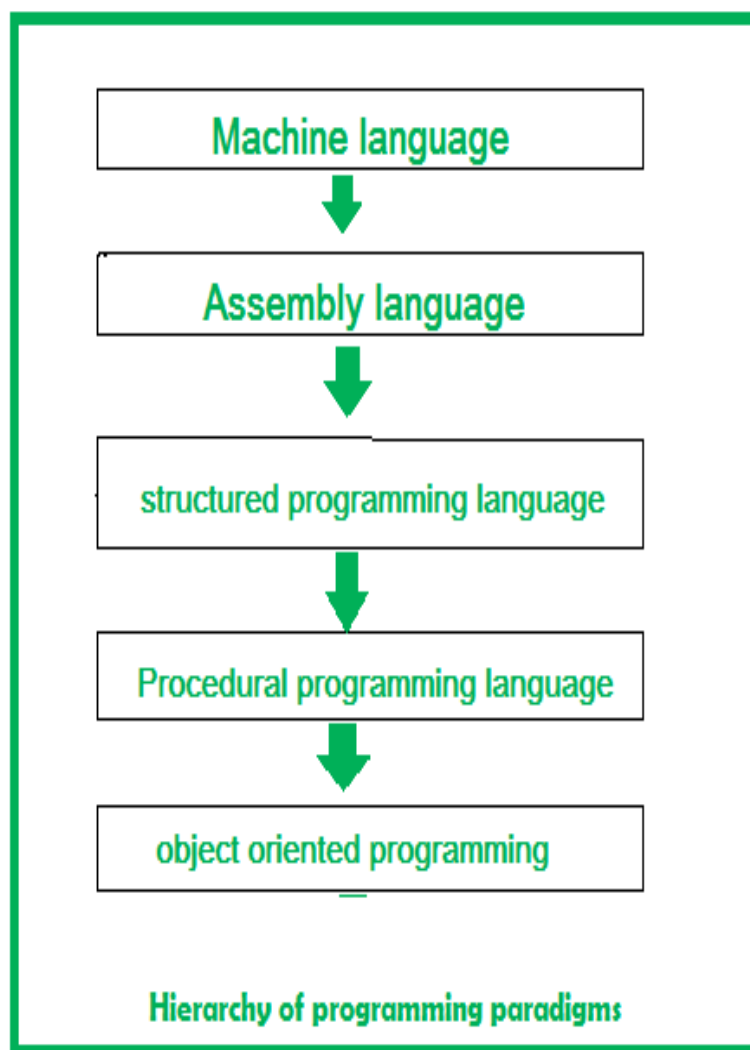
SO1.1 Define OOPS on the basis of bottom up approach SO1.2 Compare the features of procedure and Object Oriented programming SO1.3 List features of OO Programming paradigm SO1.4 Identify benefits and limitations of oops.	LE1.1 Develop simple C++ program to study elementary keywords, datatypes and logical operators in C++	Unit 1.0 Fundamentals of Object Oriented Programming 1.1 Evolution of OO Programming 1.1.1 Comparison of Procedure-Oriented and Object Oriented programming 1.2 features of Object-Oriented programming 1.2.1 Object 1.2.2 Class 1.2.3 Data Hiding and Encapsulation 1.2.4 Dynamic Binding 1.2.5 Message Passing 1.2.6 Inheritance 1.2.7 Polymorphism 1.3 Benefits of Object-Oriented programming 1.4 Applications of Object-Oriented programming 1.5 keywords in C++ like friend, const, auto, register, extern, typedef, static, default, return, void, continue 1.6 Structure of C++ program 1.7 Basic Data types & user defined data types 1.8 Various operators in C++ like Arithmetic, Relational, Logical	1.1 Identify the major differences in syntax by comparing the program structure of C and C++ for few similar programs 1.2 List all keywords in C++ 1.3 Basics of debugging
--	--	--	---

Session Outcomes (SOs)	Laboratory	Class room Instruction (CI)	Self Learning (SL)
		Scope resolution, member dereferencing operator, memory management operators, type cast, operator precedence 1.9 Manipulators	

Evolution of OOP:-

Object-oriented design started right from the moment computers were invented. Programming was there, and programming approaches came into the picture. Programming is basically giving certain instructions to the computer.

At the beginning of the computing era, programming was usually limited to machine language programming. Machine language means those sets of instructions that are specific to a particular machine or processor, which are in the form of 0's and 1's. These are sequences of bits (0100110...). But it's quite difficult to write a program or develop software in machine language. It's actually impossible to develop software used in today's scenarios with sequences of bits. This was the main reason programmers moved on to the next generation of programming languages, developing assembly languages, which were near enough to the English language to easily understand. These assembly languages were used in microprocessors. With the invention of the microprocessor, assembly languages flourished and ruled over the industry, but it was not enough. Again, programmers came up with something new, i.e., structured and procedural programming.



Structured Programming –

The basic principle of the structured programming approach is to divide a program into functions and modules.

It is based on top down strategy. The use of modules and functions makes the program more understandable and readable. It helps to write cleaner code and to maintain control over the functions and modules. This approach gives importance to functions rather than data. It focuses on the development of large software applications, for example, C was used for modern operating system development. The programming languages: PASCAL (introduced by Niklaus Wirth) and C (introduced by Dennis Ritchie) follow this approach.

Structured Programming, as name suggests, is a technique that is considered as precursor to OOP and usually consists of well-structured and separated modules. In this programming, user can create its own user-defined functions as well as this methodology tries to resolve issues that are associated with unconditional transfers to allow programmers follow logic of programs. It also requires more discipline at the design and logical structuring stage.

Example : Pascal, ALGOL, C, Modula-2, etc.

Procedural Programming Approach.

This approach is also known as the top-down approach. In this approach, a program is divided into functions that perform specific tasks.

It is defined as a programming language derived from the structure programming and based on calling procedures. The procedures are the functions, routines, or subroutines that consist of the computational steps required to be carried. It follows a step-by-step approach in order to break down a task into a set of variables and routines via a sequence of instructions.

This approach is mainly used for medium-sized applications. Data is global, and all the functions can access global data. The basic drawback of the procedural programming approach is that data is not secured because data is global and can be accessed by any function. Program control flow is achieved through function calls and goto statements. The programming languages: FORTRAN (developed by IBM) and COBOL (developed by Dr. Grace Murray Hopper) follow this approach. These programming constructs were developed in the late 1970s and 1980s. There were still some issues with these languages, though they fulfilled the criteria of well-structured programs, software, etc. They were not as structured as the requirements were at that time. They seem to be over-generalized and don't correlate with real-time applications. To solve such kinds of problems, OOP, an object-oriented approach was developed as a solution.

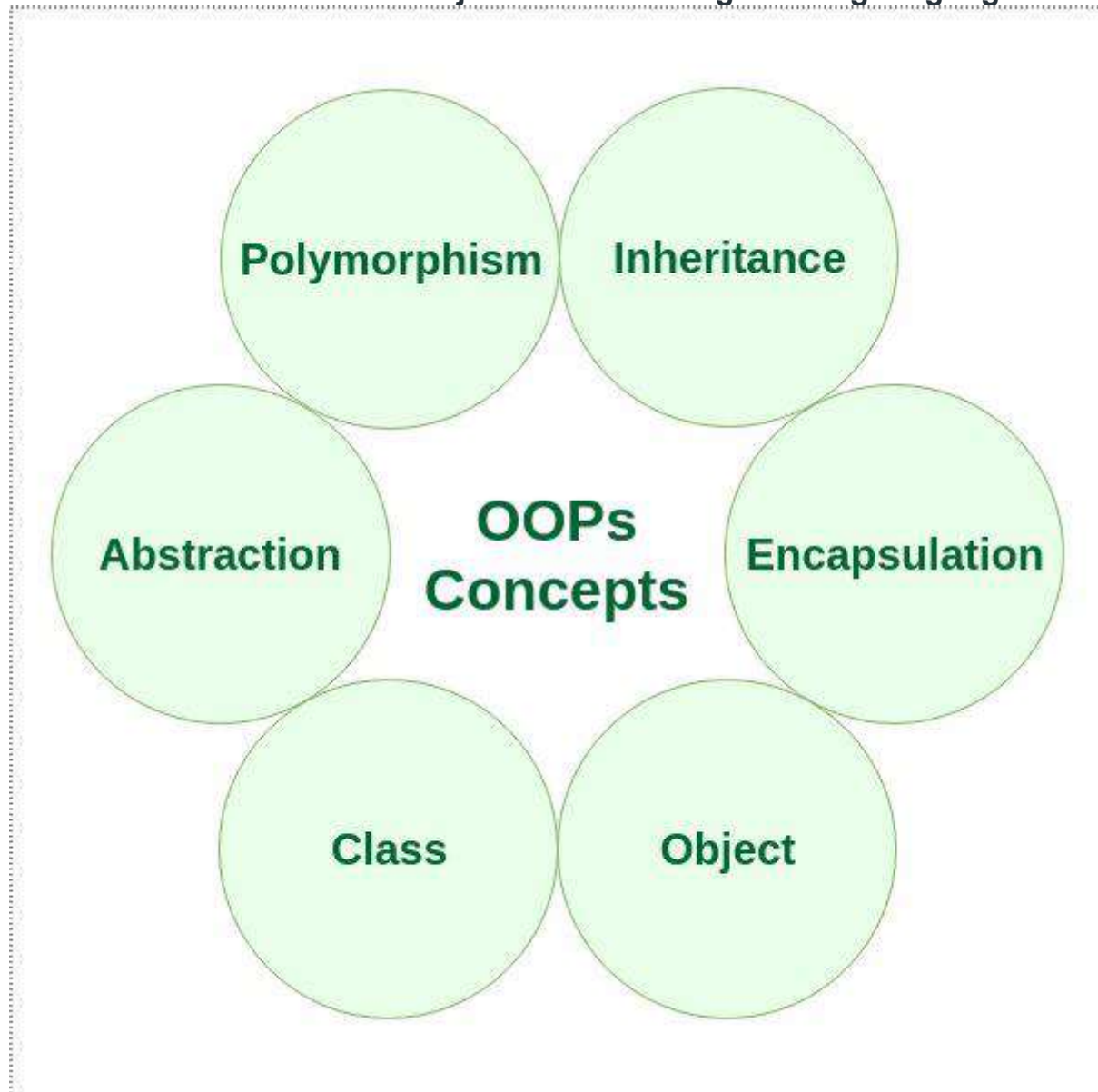
The difference between both Procedural oriented and object oriented programming languages are tabulated as follows –

Procedural Oriented Programming	Object Oriented Programming
In procedural programming, program is divided into small parts called <i>functions</i> .	In object oriented programming, program is divided into small parts called <i>objects</i> .
Procedural programming follows <i>top down approach</i> .	Object oriented programming follows <i>bottom up approach</i> .
There is no access specifier in procedural programming.	Object oriented programming have access specifiers like private, public, protected etc.
Adding new data and function is not easy.	Adding new data and function is easy.
Procedural programming does not have any proper way for hiding data so it is <i>less secure</i> .	Object oriented programming provides data hiding so it is <i>more secure</i> .
In procedural programming, overloading is not possible.	Overloading is possible in object oriented programming.
In procedural programming, function is more important than data.	In object oriented programming, data is more important than function.
Procedural programming is based on <i>unreal world</i> .	Object oriented programming is based on <i>real world</i> .
Examples: C, FORTRAN, Pascal, Basic etc.	Examples: C++, Java, Python, C# etc.

Object-oriented programming –

As the name suggests uses [objects](#) in programming. Object-oriented programming aims to implement real-world entities like inheritance, hiding, polymorphism, etc in programming. The main aim of OOP is to bind together the data and the functions that operate on them so that no other part of the code can access this data except that function.

Characteristics of an Object Oriented Programming language



Class: The building block of C++ that leads to Object-Oriented programming is a Class. It is a user-defined data type, which holds its own data members and member functions, which can be accessed and used by creating an instance of that class. A class is like a blueprint for an object.

For Example: Consider the Class of Cars. There may be many cars with different names and brand but all of them will share some common

properties like all of them will have 4 wheels, Speed Limit, Mileage range etc. So here, Car is the class and wheels, speed limits, mileage are their properties.

- A Class is a user-defined data-type which has data members and member functions.
- Data members are the data variables and member functions are the functions used to manipulate these variables and together these data members and member functions define the properties and behaviour of the objects in a Class.
- In the above example of class Car, the data member will be speed limit, mileage etc and member functions can apply brakes, increase speed etc.

We can say that a **Class in C++** is a blue-print representing a group of objects which shares some common properties and behaviours.

Object: An Object is an identifiable entity with some characteristics and behaviour. An Object is an instance of a Class. When a class is defined, no memory is allocated but when it is instantiated (i.e. an object is created) memory is allocated.

Object take up space in memory and have an associated address like a record in pascal or structure or union in C.

When a program is executed the objects interact by sending messages to one another.

Each object contains data and code to manipulate the data. Objects can interact without having to know details of each other's data or code, it is sufficient to know the type of message accepted and type of response returned by the objects.

Prototype:

```
class person
{
    char name[20];
    int id;
public:
    void getdetails(){}
};

int main()
{
    person p1; // p1 is a object
}
```

Encapsulation: In normal terms, Encapsulation is defined as wrapping up of data and information under a single unit. In Object-Oriented Programming, Encapsulation is defined as binding together the data and the functions that manipulate them.

Consider a real-life example of encapsulation, in a company, there are different sections like the accounts section, finance section, sales section

etc. The finance section handles all the financial transactions and keeps records of all the data related to finance. Similarly, the sales section handles all the sales-related activities and keeps records of all the sales. Now there may arise a situation when for some reason an official from the finance section needs all the data about sales in a particular month. In this case, he is not allowed to directly access the data of the sales section. He will first have to contact some other officer in the sales section and then request him to give the particular data. This is what encapsulation is. Here the data of the sales section and the employees that can manipulate them are wrapped under a single name “sales section”.

Encapsulation in C++



Encapsulation in C++

Encapsulation also leads to *data abstraction or hiding*. As using encapsulation also hides the data. In the above example, the data of any of the section like sales, finance or accounts are hidden from any other section.

Abstraction: Data abstraction is one of the most essential and important features of object-oriented programming in C++. Abstraction means displaying only essential information and hiding the details. Data abstraction refers to providing only essential information about the data to the outside world, hiding the background details or implementation.

Consider a real-life example of a man driving a car. The man only knows that pressing the accelerators will increase the speed of the car or applying brakes will stop the car but he does not know about how on pressing accelerator the speed is actually increasing, he does not know about the inner mechanism of the car or the implementation of accelerator, brakes etc in the car. This is what abstraction is.

- *Abstraction using Classes:* We can implement Abstraction in C++ using classes. The class helps us to group data members and member functions using available access specifiers. A Class can decide which data member will be visible to the outside world and which is not.
- *Abstraction in Header files:* One more type of abstraction in C++ can be header files. For example, consider the `pow()` method present in `math.h` header file. Whenever we need to calculate the power of a number, we simply call the function `pow()` present in the `math.h` header file and pass the numbers as arguments without knowing the underlying algorithm according to which the function is actually calculating the power of numbers.

Polymorphism: The word polymorphism means having many forms. In simple words, we can define polymorphism as the ability of a message to be displayed in more than one form.

A person at the same time can have different characteristics. Like a man at the same time is a father, a husband, an employee. So the same person possesses different behaviour in different situations. This is called polymorphism.

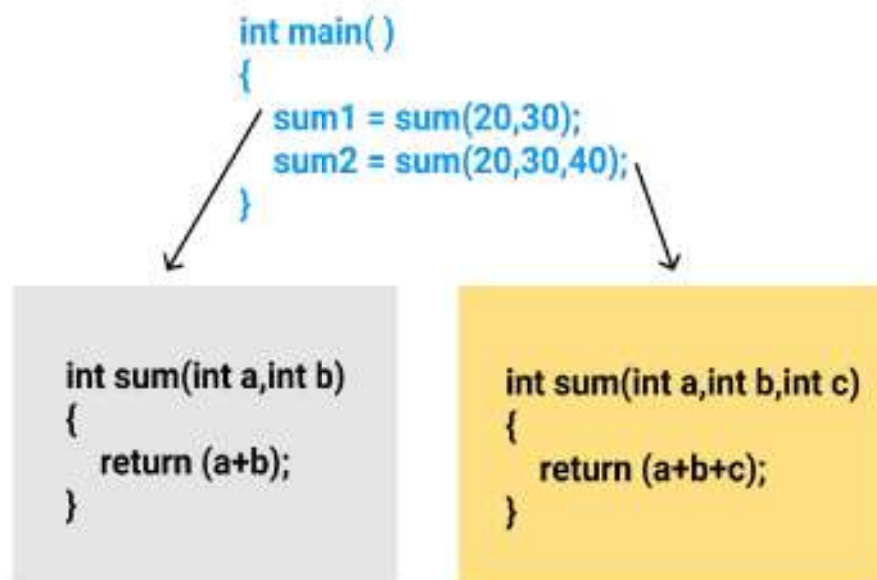
An operation may exhibit different behaviours in different instances. The behaviour depends upon the types of data used in the operation.

C++ supports operator overloading and function overloading.

- *Operator Overloading:* The process of making an operator to exhibit different behaviours in different instances is known as operator overloading.
- *Function Overloading:* Function overloading is using a single function name to perform different types of tasks.

Polymorphism is extensively used in implementing inheritance.

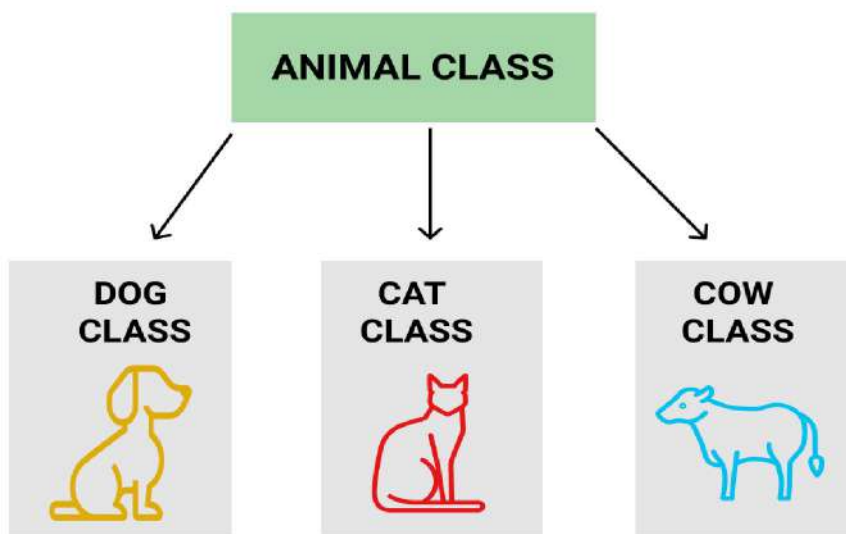
Example: Suppose we have to write a function to add some integers, some times there are 2 integers, some times there are 3 integers. We can write the Addition Method with the same name having different parameters, the concerned method will be called according to parameters.



Inheritance: The capability of a class to derive properties and characteristics from another class is called Inheritance. Inheritance is one of the most important features of Object-Oriented Programming.

- **Sub Class:** The class that inherits properties from another class is called Sub class or Derived Class.
- **Super Class:** The class whose properties are inherited by sub class is called Base Class or Super class.
- **Reusability:** Inheritance supports the concept of “reusability”, i.e. when we want to create a new class and there is already a class that includes some of the code that we want, we can derive our new class from the existing class. By doing this, we are reusing the fields and methods of the existing class.

Example: Dog, Cat, Cow can be Derived Class of Animal Base Class.



Data hiding:-

It is a software development technique specifically used in object-oriented programming (OOP) to **hide** internal object details (**data** members). **Data hiding** ensures exclusive **data** access to class members and protects object integrity by preventing unintended or intended changes.

Dynamic Binding: In dynamic binding, the code to be executed in response to function call is decided at runtime.

Message Passing: Objects communicate with one another by sending and receiving information to each other. A message for an object is a request for execution of a procedure and therefore will invoke a function in the receiving object that generates the desired results. Message passing involves specifying the name of the object, the name of the function and the information to be sent.

Class: A class in C++ is the building block that leads to Object-Oriented programming. It is a user-defined data type, which holds its own data members and member functions, which can be accessed and used by

creating an instance of that class. A C++ class is like a blueprint for an object.

For Example: Consider the Class of **Cars**. There may be many cars with different names and brand but all of them will share some common properties like all of them will have *4 wheels, Speed Limit, Mileage range* etc. So here, Car is the class and wheels, speed limits, mileage are their properties.

- A Class is a user defined data-type which has data members and member functions.
- Data members are the data variables and member functions are the functions used to manipulate these variables and together these data members and member functions defines the properties and behavior of the objects in a Class.
- In the above example of class *Car*, the data member will be *speed limit, mileage* etc and member functions can be *apply brakes, increase speed* etc.

An **Object** is an instance of a Class. When a class is defined, no memory is allocated but when it is instantiated (i.e. an object is created) memory is allocated.

Defining Class and Declaring Objects

A class is defined in C++ using keyword `class` followed by the name of class. The body of class is defined inside the curly brackets and terminated by a semicolon at the end.

keyword user-defined name

```
class ClassName
{
    Access specifier:           //can be private,public or protected
    Data members;              // Variables to be used
    Member Functions() { }     //Methods to access data members
};                             // Class name ends with a semicolon
```

Declaring Objects: When a class is defined, only the specification for the object is defined; no memory or storage is allocated. To use the data and access functions defined in the class, you need to create objects.

Syntax:

`ClassName ObjectName;`

Accessing data members and member functions: The data members and member functions of class can be accessed using the dot('.') operator with

the object. For example if the name of object is *obj* and you want to access the member function with the name *printName()* then you will have to write *obj.printName()* .

Accessing Data Members

The public data members are also accessed in the same way given however the private data members are not allowed to be accessed directly by the object. Accessing a data member depends solely on the access control of that data member.

This access control is given by [Access modifiers in C++](#). There are three access modifiers : **public**, **private** and **protected**.

```
// C++ program to demonstrate
// accessing of data members
#include <bits/stdc++.h>
using namespace std;
class Geeks
{
    // Access specifier
    public:
    // Data Members
    string geekname;
    // Member Functions()
    void printname()
    {
        cout << "Geekname is: " << geekname;
    }
};
int main() {
    // Declare an object of class geeks
    Geeks obj1;
    // accessing data member
    obj1.geekname = "Abhi";
    // accessing member function
    obj1.printname();
    return 0;
}
```

Output:

Geekname is: Abhi

Access Modifiers in C++

Access modifiers are used to implement an important aspect of Object-Oriented Programming known as Data Hiding.

There are 3 types of access modifiers available in C++:

1. **Public**
2. **Private**
3. **Protected**

Public: All the class members declared under the public specifier will be available to everyone. The data members and member functions declared as public can be accessed by other classes and functions too. The public members of a class can be accessed from anywhere in the program using the direct member access operator (.) with the object of that class.

```
// C++ program to demonstrate public
// access modifier
1. #include<iostream>
2. using namespace std;
3. // class definition
4. class Circle
5. {
6.     public:
7.         double radius;
8.         double compute_area()
9.         {
10.             return 3.14*radius*radius;
11.         }
12.     };
13. // main function
14. int main()
15. {
16.     Circle obj;
17.
18.     // accessing public datamember outside class
19.     obj.radius = 5.5;
20.     cout << "Radius is: " << obj.radius << "\n";
21.     cout << "Area is: " << obj.compute_area();
22.     return 0;
23. }
```

Output:-

Radius is: 5.5

Area is: 94.985

In the above program the data member *radius* is declared as public so it could be accessed outside the class and thus was allowed access from inside main().

Private: The class members declared as *private* can be accessed only by the member functions inside the class. They are not allowed to be accessed directly by any object or function outside the class. Only the member

functions or the friend functions are allowed to access the private data members of a class.

```
// C++ program to demonstrate private
// access modifier

#include<iostream>
using namespace std;

class Circle
{
    // private data member
    private:
        double radius;

    // public member function
    public:
        double compute_area()
        {    // member function can access private
            // data member radius
            return 3.14*radius*radius;
        }
};

// main function
int main()
{
    // creating object of the class
    Circle obj;

    // trying to access private data member
    // directly outside the class
    obj.radius = 1.5;

    cout << "Area is:" << obj.compute_area();
    return 0;
}
```

Output:

In function 'int main()':

11:16: error: 'double Circle::radius' is private

double radius;

^

31:9: error: within this context

obj.radius = 1.5;

^

The output of above program is a compile time error because we are not allowed to access the private data members of a class directly outside the class. Yet an access to obj.radius is attempted, radius being a private data member we obtain a compilation error.

However, we can access the private data members of a class indirectly using the public member functions of the class.

```
/ C++ program to demonstrate private
// access modifier

#include<iostream>
using namespace std;

class Circle
{
    // private data member
    private:
        double radius;

    // public member function
    public:
        void compute_area(double r)
        {    // member function can access private
            radius = r;

            double area = 3.14*radius*radius;

            cout << "Radius is: " << radius << endl;
            cout << "Area is: " << area;
        }
};

int main()
{
    // creating object of the class
    Circle obj;

    // trying to access private data member
    // directly outside the class
    obj.compute_area(1.5);
    return 0;
}
```

Output:-

Radius is: 1.5

Area is: 7.065

3. Protected: Protected access modifier is similar to private access modifier in the sense that it can't be accessed outside of it's class unless with the help of friend class, the difference is that the class members declared as Protected can be accessed by any subclass(derived class) of that class as well.

Note: This access through inheritance can alter the access modifier of the elements of base class in derived class depending on the [modes of Inheritance](#).

```
// C++ program to demonstrate
// protected access modifier
#include <bits/stdc++.h>
using namespace std;

// base class
```



```

class Parent
{
    // protected data members
    protected:
    int id_protected;

};

// sub class or derived class from public base class
class Child : public Parent
{
    public:
    void setId(int id)
    {

        // Child class is able to access the inherited
        // protected data members of base class

        id_protected = id;

    }

    void displayId()
    {
        cout << "id_protected is: " << id_protected << endl;
    }
};

// main function
int main() {
    Child obj1;
    // member function of the derived class can
    // access the protected data members of the base class
    obj1.setId(81);
    obj1.displayId();
    return 0;
}

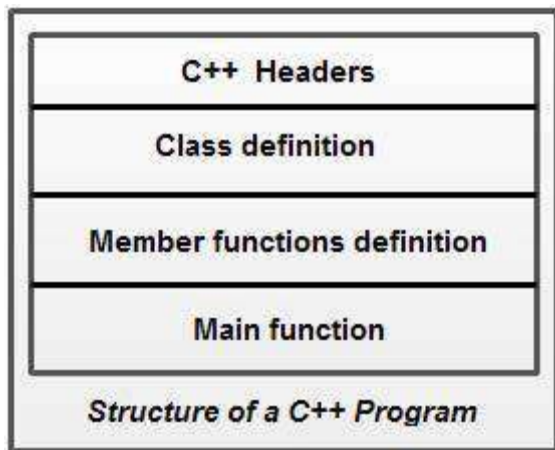
```

Output:

id_protected is: 81

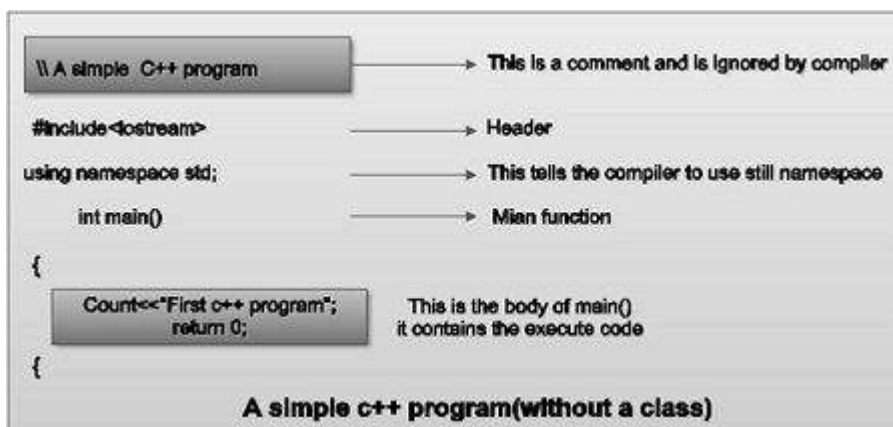
Structure of a C++ Program

Programs are a sequence of instructions or statements. These statements form the structure of a C++ program. C++ program structure is divided into various sections, namely, *headers*, *class definition*, *member functions definitions* and *main function*.



Note that C++ provides the flexibility of writing a program with or without a class and its member functions definitions. A simple C++ program (without a class) includes comments, headers, namespace, main() and input/output statements.

Comments are a vital element of a program that is used to increase the readability of a program and to describe its functioning. Comments are not executable statements and hence, do not increase the size of a file.



C++ supports two comment styles: single line comment and multiline comment. Single line comments are used to define line-by-line descriptions. Double slash (//) is used to represent single line comments. To understand the concept of single line comment, consider this statement.

Headers: Generally, a program includes various programming elements like built-in functions, classes, keywords, constants, operators, etc., that are already defined in the standard C++ library. In order to use such pre-defined elements in a program, an appropriate header must be included in the program. The standard headers contain the [information](#) like prototype, definition and return type of library functions, [data type](#) of constants, etc. As a result, programmers do not need to explicitly declare (or define) the predefined programming elements.

Standard headers are specified in a program through the preprocessor directive `#include`. In Figure, the `iostream` header is used. When the compiler processes the instruction `#include<iostream>`, it includes the contents of `iostream` in the program. This enables the programmer to use *standard input*, *output* and *error* facilities that are provided only through the standard streams defined in `<iostream>`. These standard streams process data as a stream of characters, that is, data is read and

displayed in a continuous flow. The standard streams defined in <iostream> are listed here.

Namespace: Since its creation, C++ has gone through many changes by the C++ Standards Committee. One of the new features added to this language is namespace. A namespace permits grouping of various entities like classes, objects, functions and various C++ tokens, etc., under a single name. Different users can create separate namespaces and thus can use similar names of the entities. This avoids compile-time error that may exist due to identical-name conflicts.

The C++ Standards Committee has rearranged the entities of the standard library under a namespace called std. In Figure, the statement using namespace std informs the compiler to include all the entities present in the namespace std. The entities of a namespace can be accessed in different ways which are listed here.

- By specifying the using directive

using namespace std;

cout<<“Hello World”;

- By specifying the full member name

std: :cout<<“Hello World”;

- By specifying the using declaration

using std:: cout;

cout<<“Hello World”;

As soon as the new-style header is included, its contents are included in the std namespace. Thus, all the modern C++ compilers support these statements.

#include<iostream>

using namespace std;

Main Function: The main () is a startup function that starts the execution of a c++ program. All C++ statements that need to be executed are written within main (). The compiler executes all the instructions written within the opening and closing curly braces ' {} ' that enclose the body of main (). Once all the instructions in main () are executed, the control passes out of main (), terminating the entire program and returning a value to the [operating system](#).

By default, main () in C++ returns an int value to the [operating system](#). Therefore, main () should end with the return 0 statement. A return value zero indicates success and a non-zero value indicates failure or error.

Class Definition:

The class contains members of the class and the members can be data member and member functions.

Functions and data are the members of the class. Functions can be defined inside of the class or outside of the class.

Operators in C / C++

Operators are the foundation of any programming language. We can define operators as symbols that help us to perform specific mathematical and logical computations on operands. In other words, we can say that an operator operates the operands. For example, '+' is an operator used for addition, as shown below:

```
c = a + b;
```

Here, '+' is the operator known as the addition operator and 'a' and 'b' are operands. The addition operator tells the compiler to add both of the operands 'a' and 'b'.

The functionality of the C/C++ programming language is incomplete without the use of operators.

C/C++ has many built-in operators and can be classified into 6 types:

1. Arithmetic Operators
2. Relational Operators
3. Logical Operators
4. Bitwise Operators
5. Assignment Operators

1. Arithmetic Operators:

These operators are used to perform arithmetic/mathematical operations on operands. Examples: (+, -, *, /, %, ++, --). Arithmetic operators are of two types:

a) Unary Operators: Operators that operate or work with a single operand are unary operators. For example: Increment(++), and Decrement(--) Operators

```
int val = 5;
```

```
++val; // 6
```

b) Binary Operators: Operators that operate or work with two operands are binary operators. For example: Addition(+), Subtraction(-), multiplication(*), Division(/) operators

```
int a = 7;
```

```
int b = 2;
```

```
cout<<a+b; // 9
```

Arithmetic Operators

There are following arithmetic operators supported by C++ language –

Assume variable A holds 10 and variable B holds 20, then –

Show Examples

Operator	Description	Example
+	Adds two operands	A + B will give 30

-	Subtracts second operand from the first	A - B will give -10
*	Multiplies both operands	A * B will give 200
/	Divides numerator by de-numerator	B / A will give 2
%	Modulus Operator and remainder of after an integer division	B % A will give 0
++	<u>Increment operator</u> , increases integer value by one	A++ will give 11
--	<u>Decrement operator</u> , decreases integer value by one	A-- will give 9

Relational Operators

There are following relational operators supported by C++ language

Assume variable A holds 10 and variable B holds 20, then –

Show Examples

Operator	Description	Example
==	Checks if the values of two operands are equal or not, if yes then condition becomes true.	(A == B) is not true.
!=	Checks if the values of two operands are equal or not, if values are not equal then condition becomes true.	(A != B) is true.
>	Checks if the value of left operand is greater than the value of right operand, if yes then condition becomes true.	(A > B) is not true.

<	Checks if the value of left operand is less than the value of right operand, if yes then condition becomes true.	(A < B) is true.
>=	Checks if the value of left operand is greater than or equal to the value of right operand, if yes then condition becomes true.	(A >= B) is not true.
<=	Checks if the value of left operand is less than or equal to the value of right operand, if yes then condition becomes true.	(A <= B) is true.

Logical Operators

There are following logical operators supported by C++ language.

Assume variable A holds 1 and variable B holds 0, then –

Show Examples

Operator	Description	Example
&&	Called Logical AND operator. If both the operands are non-zero, then condition becomes true.	(A && B) is false.
	Called Logical OR Operator. If any of the two operands is non-zero, then condition becomes true.	(A B) is true.
!	Called Logical NOT Operator. Use to reverses the logical state of its operand. If a condition is true, then Logical NOT operator will make false.	!(A && B) is true.

Bitwise Operators

Bitwise operator works on bits and perform bit-by-bit operation. The truth tables for &, |, and ^ are as follows –

p	q	p & q	p q	p ^ q
0	0	0	0	0
0	1	0	1	1
1	1	1	1	0
1	0	0	1	1

Assume if A = 60; and B = 13; now in binary format they will be as follows –

A = 0011 1100

B = 0000 1101

A&B = 0000 1100

A|B = 0011 1101

A^B = 0011 0001

~A = 1100 0011

The Bitwise operators supported by C++ language are listed in the following table. Assume variable A holds 60 and variable B holds 13, then –

Show Examples

Operator	Description	Example
&	Binary AND Operator copies a bit to the result if it exists in both operands.	(A & B) will give 12 which is 0000 1100
	Binary OR Operator copies a bit if it exists in either operand.	(A B) will give 61 which is 0011 1101

<code>^</code>	Binary XOR Operator copies the bit if it is set in one operand but not both.	$(A \wedge B)$ will give 49 which is 0011 0001
<code>~</code>	Binary Ones Complement Operator is unary and has the effect of 'flipping' bits.	$(\sim A)$ will give -61 which is 1100 0011 in 2's complement form due to a signed binary number.
<code><<</code>	Binary Left Shift Operator. The left operands value is moved left by the number of bits specified by the right operand.	$A \ll 2$ will give 240 which is 1111 0000
<code>>></code>	Binary Right Shift Operator. The left operands value is moved right by the number of bits specified by the right operand.	$A \gg 2$ will give 15 which is 0000 1111

Assignment Operators

There are following assignment operators supported by C++ language –

Show Examples

Operator	Description	Example
<code>=</code>	Simple assignment operator, Assigns values from right side operands to left side operand.	$C = A + B$ will assign value of $A + B$ into C
<code>+=</code>	Add AND assignment operator, It adds right operand to the left operand and assign the result to left operand.	$C += A$ is equivalent to $C = C + A$
<code>-=</code>	Subtract AND assignment operator, It subtracts right operand from the left operand and assign the result to left operand.	$C -= A$ is equivalent to $C = C - A$
<code>*=</code>	Multiply AND assignment operator, It multiplies right operand with the left operand and assign the result to left operand.	$C *= A$ is equivalent to $C = C * A$

/=	Divide AND assignment operator, It divides left operand with the right operand and assign the result to left operand.	C /= A is equivalent to C = C / A
%=	Modulus AND assignment operator, It takes modulus using two operands and assign the result to left operand.	C %= A is equivalent to C = C % A
<<=	Left shift AND assignment operator.	C <<= 2 is same as C = C << 2
>>=	Right shift AND assignment operator.	C >>= 2 is same as C = C >> 2
&=	Bitwise AND assignment operator.	C &= 2 is same as C = C & 2
^=	Bitwise exclusive OR and assignment operator.	C ^= 2 is same as C = C ^ 2
=	Bitwise inclusive OR and assignment operator.	C = 2 is same as C = C 2

Misc Operators

The following table lists some other operators that C++ supports.

Sr.No	Operator & Description
1	sizeof <u>sizeof operator</u> returns the size of a variable. For example, sizeof(a), where 'a' is integer, and will return 4.
2	Condition ? X : Y <u>Conditional operator (?)</u> . If Condition is true then it returns value of X otherwise returns value of Y.
3	, <u>Comma operator</u> causes a sequence of operations to be performed. The value of

	the entire comma expression is the value of the last expression of the comma-separated list.
4	. (dot) and -> (arrow) <u>Member operators</u> are used to reference individual members of classes, structures, and unions.
5	Cast <u>Casting operators</u> convert one data type to another. For example, int(2.2000) would return 2.
6	& <u>Pointer operator &</u> returns the address of a variable. For example &a; will give actual address of the variable.
7	* <u>Pointer operator *</u> is pointer to a variable. For example *var; will pointer to a variable var.

Operators Precedence in C++

Operator precedence determines the grouping of terms in an expression. This affects how an expression is evaluated. Certain operators have higher precedence than others; for example, the multiplication operator has higher precedence than the addition operator –

For example $x = 7 + 3 * 2$; here, x is assigned 13, not 20 because operator $*$ has higher precedence than $+$, so it first gets multiplied with $3*2$ and then adds into 7.

Here, operators with the highest precedence appear at the top of the table, those with the lowest appear at the bottom. Within an expression, higher precedence operators will be evaluated first.

Precedence	Operator	Description	Associativity
1.	()	Parentheses (function call)	left-to-right
	[]	Brackets (array subscript)	
	.	Member selection via object name	
	->	Member selection via a pointer	
	++/--	Postfix increment/decrement	
2.	++/--	Prefix increment/decrement	right-to-left
	+/-	Unary plus/minus	
	!~	Logical negation/bitwise complement	
	(type)	Cast (convert value to temporary value	

		of type)	
	*	Dereference	
	&	Address (of operand)	
	sizeof	Determine size in bytes on this implementation	
3.	*,/,%	Multiplication/division/modulus	left-to-right
4.	+/-	Addition/subtraction	left-to-right
5.	<< , >>	Bitwise shift left, Bitwise shift right	left-to-right
6.	< , <=	Relational less than/less than or equal to	left-to-right
	> , >=	Relational greater than/greater than or equal to	left-to-right
7.	== , !=	Relational is equal to/is not equal to	left-to-right
8.	&	Bitwise AND	left-to-right
9.	^	Bitwise exclusive OR	left-to-right
10.		Bitwise inclusive OR	left-to-right
11.	&&	Logical AND	left-to-right
12.		Logical OR	left-to-right
13.	?:	Ternary conditional	right-to-left
14.	=	Assignment	right-to-left
	+= , -=	Addition/subtraction assignment	
	*= , /=	Multiplication/division assignment	
	%= , &=	Modulus/bitwise AND assignment	
	^= , =	Bitwise exclusive/inclusive OR assignment	
	<>=	Bitwise shift left/right assignment	
15.	,	expression separator	left-to-right

Memory Management Operators

In [C language](#), we use the `malloc()` or `calloc()` functions to allocate the memory dynamically at run time, and `free()` function is used to deallocate the dynamically allocated memory. [C++](#) also supports these functions, but C++ also defines unary operators such as `new` and `delete` to perform the same tasks, i.e., allocating and freeing the memory.

New operator

A `new` operator is used to create the object while a `delete` operator is used to delete the object. When the object is created by using the `new` operator, then the object will exist until we explicitly use the `delete` operator to delete the object. Therefore, we can say that the lifetime of the object is not related to the block structure of the program.

Syntax

```
pointer_variable = new data-type
```

The above syntax is used to create the object using the new operator. In the above syntax, '**pointer_variable**' is the name of the pointer variable, '**new**' is the operator, and '**data-type**' defines the type of the data.

```
int *p;  
p = new int;
```

In the above example, 'p' is a pointer of type int.

Delete operator

When memory is no longer required, then it needs to be deallocated so that the memory can be used for another purpose. This can be achieved by using the delete operator, as shown below:

```
delete pointer_variable;
```

In the above statement, '**delete**' is the operator used to delete the existing object, and '**pointer_variable**' is the name of the pointer variable.

In the previous case, we have created two pointers 'p' and 'q' by using the new operator, and can be deleted by using the following statements:

```
delete p;  
delete q;
```

The dynamically allocated array can also be removed from the memory space by using the following syntax:

```
delete [size] pointer_variable;
```

In the above statement, we need to specify the size that defines the number of elements that are required to be freed. The drawback of this syntax is that we need to remember the size of the array. But, in recent versions of C++, we do not need to mention the size as follows:

```
delete [ ] pointer_variable;
```

1. `#include <iostream>`
2. `using namespace std`
3. `int main()`
4. `{`
5. `int size; // variable declaration`
6. `int *arr = new int[size]; // creating an array`
7. `cout<<"Enter the size of the array : ";`
8. `std::cin >> size; //`
9. `cout<<"\nEnter the element : ";`
10. `for(int i=0;i<size;i++) // for loop`
11. `{`
12. `cin>>arr[i];`

```

13. }
14. cout<<"\nThe elements that you have entered are :";
15. for(int i=0;i<size;i++) // for loop
16. {
17. cout<<arr[i]<<" ";
18. }
19. delete arr; // deleting an existing array.
20. return 0;
21. }

```

Type Casting in C++

This section will discuss the type casting of the variables in the C++ programming language. Type casting refers to the conversion of one data type to another in a program. Typecasting can be done in two ways: automatically by the compiler and manually by the programmer or user. Type Casting is also known as Type Conversion.

For example, suppose the given data is an integer type, and we want to convert it into float type. So, we need to manually cast int data to the float type, and this type of casting is called the Type Casting in C++.

1. **int** num = 5;
2. **float** x;
3. x = **float**(num);
4. Output, x = 5.0

2nd example:

1. **float** num = 5.25;
2. **int** x;
3. x = **int**(num);
4. Output, x = 5

Type Casting is divided into two types: Implicit conversion or Implicit Type Casting and Explicit Type Conversion or Explicit Type Casting.

Implicit Type Casting or Implicit Type Conversion

- It is known as the automatic type casting.

- It automatically converted from one data type to another without any external intervention such as programmer or user. It means the compiler automatically converts one data type to another.
- All data type is automatically upgraded to the largest type without losing any information.
- It can only apply in a program if both variables are compatible with each other.

char - sort **int** -> **int** -> unsigned **int** -> **long int** -> **float** -> **double**-> **long double**, etc.

Program to use the implicit type casting in C++

Let's create an example to demonstrate the casting of one variable to another using the implicit type casting in C++.

```

1. #include <iostream>
2. using namespace std;
3. int main ()
4. {
5.     short x = 200;
6.     int y;
7.     y = x;
8.     cout << " Implicit Type Casting " << endl;
9.     cout << " The value of x: " << x << endl;
10.    cout << " The value of y: " << y << endl;
11.
12.    int num = 20;
13.    char ch = 'a';
14.    int res = 20 + 'a';
15.    cout << " Type casting char to int data type ('a' to 20): " << res << endl;
16.
17.    float val = num + 'A';
18.    cout << " Type casting from int data to float type: " << val << endl;
19.    return 0;
20. }
```

Output:-

```

Implicit Type Casting
The value of x: 200
The value of y: 200
Type casting char to int data type ('a' to 20): 117
```

Explicit Type Casting or Explicit Type Conversion

- It is also known as the manual type casting in a program.
- It is manually cast by the programmer or user to change from one data type to another type in a program. It means a user can easily cast one data to another according to the requirement in a program.
- It does not require checking the compatibility of the variables.
- In this casting, we can upgrade or downgrade the data type of one variable to another in a program.
- It uses the cast () operator to change the type of a variable.

For example, we have a floating pointing number is 4.534, and to convert an integer value, the statement as:

1. **int** num;
2. num = (**int**) 4.534; // cast 4.534 floating value into int data type
3. cout << num

Program to demonstrate Typecasting:-

1. **#include <iostream>**
2. **using namespace** std;
3. **int** main ()
4. {
5. // declaration of the variables
6. **int** a, b;
7. **float** res;
8. a = 21;
9. b = 5;
10. cout << " Implicit Type Casting: " << endl;
11. cout << " Result: " << a / b << endl; // it loses some information
- 12.
13. cout << " \n Explicit Type Casting: " << endl;
14. // use cast () operator to convert int data to float
15. res = (**float**) 21 / 5;

```

16. cout << " The value of float variable (res): " << res << endl;
17.
18. return 0;
19.}

```

Member De-referencing Operators

The pointer related operator & and * are called as referencing and dereferencing operators. The referencing operator (&) is a unary operator and it returns the address of its operand variable. The dereferencing operator (*) is a unary operator that returns the value present at the specified address.

Consider the below program to understand use of member referencing and dereferencing operator

```

#include<iostream.h>
void main()
{
    int n= 15;
    int *ptr;
    ptr=&n;    //referencing operator
    cout<<"\nAddress of n ="<<&n;
    cout<<"\nValue in variable ptr ="<< ptr;
    cout<<"\nValue of n ="<<n;
    cout<<"\nValue using dereferencing operator="<<*ptr;
}

```

Output

```

Address of n=0x8fa5fff4
Value in variable ptr=0x8fa5fff4
Value of n=15
Value using dereferencing operator=15

```

Scope Resolution Operator in C++

The scope resolution operator is used to reference the global variable or member function that is out of scope. Therefore, we use the scope resolution operator to access the hidden variable or function of a program. The operator is represented as the double colon (::) symbol.

For example, when the global and local variable or function has the same name in a program, and when we call the variable, by default it only accesses the inner or local variable without calling the global variable. In this way, it hides the global variable or function. To overcome this situation, we use the scope resolution operator to fetch a program's hidden variable or function.

Uses of the scope resolution Operator

1. It is used to access the hidden variables or member functions of a program.
2. It defines the member function outside of the class using the scope resolution.
3. It is used to access the static variable and static function of a class.
4. The scope resolution operator is used to override function in the Inheritance.

Program based on scope resolution operator:-

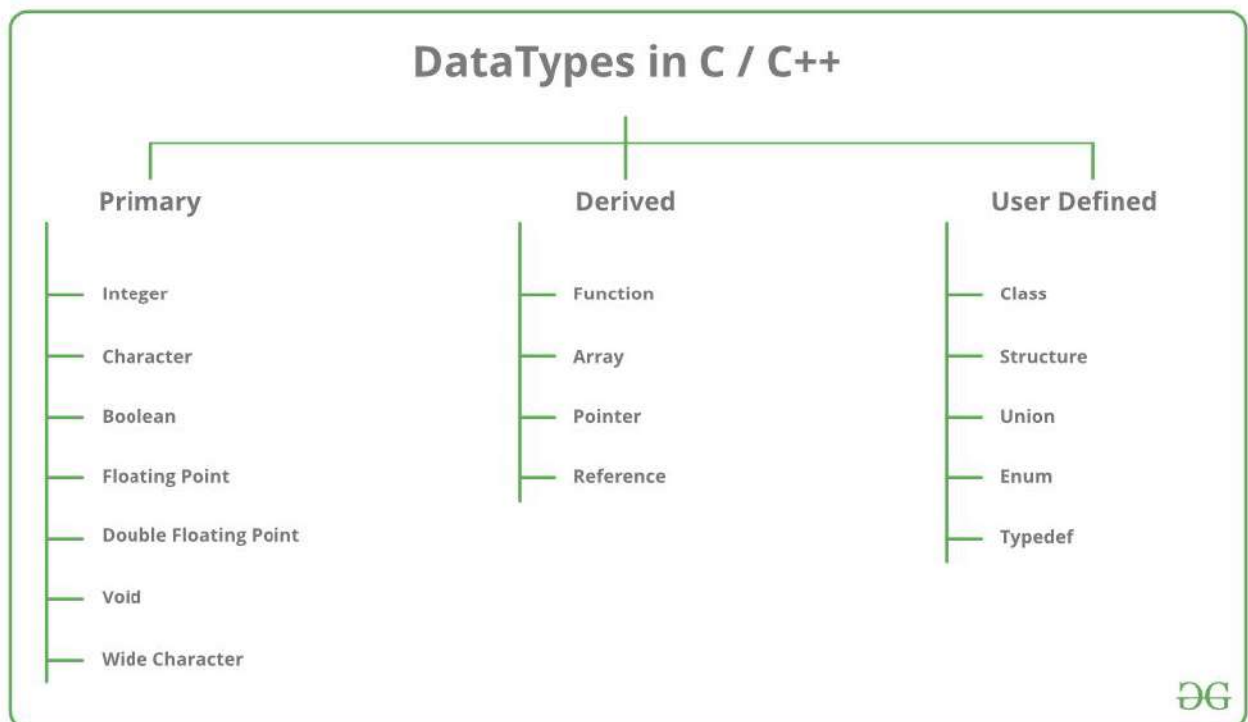

```

1. #include <iostream>
2. using namespace std;
3. // declare global variable
4. int num = 50;
5. int main ()
6. {
7. // declare local variable
8. int num = 100;
9. // print the value of the variables
10. cout << " The value of the local variable num: " << num;
11. // use scope resolution operator (::) to access the global variable
12. cout << "\n The value of the global variable num: " << ::num;
13. return 0;
14.}

```

C++ Data Types

All [variables](#) use data-type during declaration to restrict the type of data to be stored. Therefore, we can say that data types are used to tell the variables the type of data it can store. Whenever a variable is defined in C++, the compiler allocates some memory for that variable based on the data-type with which it is declared. Every data type requires a different amount of memory.



Data types in C++ is mainly divided into three types:

1. **Primitive Data Types:** These data types are built-in or predefined data types and can be used directly by the user to declare variables. example: int, char , float, bool etc. Primitive data types available in C++ are:
 - Integer
 - Character
 - Boolean
 - Floating Point
 - Double Floating Point
 - Valueless or Void
 - Wide Character
2. **Derived Data Types:** The data-types that are derived from the primitive or built-in datatypes are referred to as Derived Data Types. These can be of four types namely:
 - Function
 - Array
 - Pointer
 - Reference
3. **Abstract or User-Defined Data Types:** These data types are defined by user itself. Like, defining a class in C++ or a structure. C++ provides the following user-defined datatypes:
 - Class
 - Structure
 - Union
 - Enumeration
 - Typedef defined DataType

This article discusses **primitive data types** available in C++.

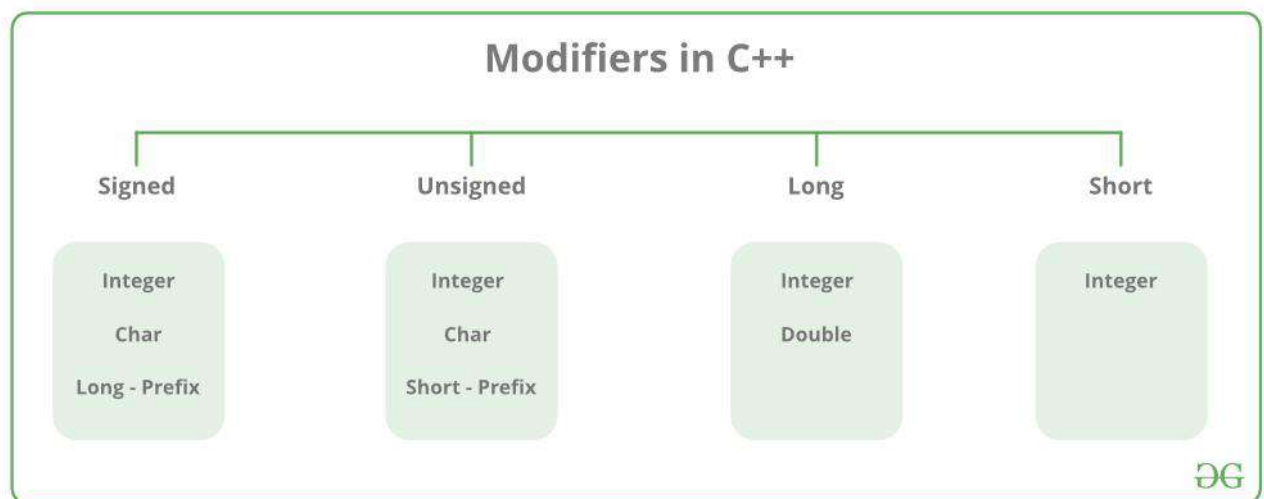
- **Integer:** Keyword used for integer data types is **int**. Integers typically requires 4 bytes of memory space and ranges from -2147483648 to 2147483647.
- **Character:** Character data type is used for storing characters. Keyword used for character data type is **char**. Characters typically requires 1 byte of memory space and ranges from -128 to 127 or 0 to 255.
- **Boolean:** Boolean data type is used for storing boolean or logical values. A boolean variable can store either *true* or *false*. Keyword used for boolean data type is **bool**.
- **Floating Point:** Floating Point data type is used for storing single precision floating point values or decimal values. Keyword used for floating point data type is **float**. Float variables typically requires 4 byte of memory space.
- **Double Floating Point:** Double Floating Point data type is used for storing double precision floating point values or decimal values. Keyword used for double

floating point data type is **double**. Double variables typically requires 8 byte of memory space.

- **void**: Void means without any value. void datatype represents a valueless entity. Void data type is used for those function which does not returns a value.
- **Wide Character**: Wide character data type is also a character data type but this data type has size greater than the normal 8-bit datatype. Represented by **wchar_t**. It is generally 2 or 4 bytes long.

Datatype Modifiers

As the name implies, datatype modifiers are used with the built-in data types to modify the length of data that a particular data type can hold.



Data type modifiers available in C++ are:

- **Signed**
- **Unsigned**
- **Short**
- **Long**

Below table summarizes the modified size and range of built-in datatypes when combined with the type modifiers:

Data Type	Size (in bytes)	Range
short int	2	-32,768 to 32,767
unsigned short int	2	0 to 65,535

unsigned int	4	0 to 4,294,967,295
Int	4	-2,147,483,648 to 2,147,483,647
long int	4	-2,147,483,648 to 2,147,483,647
unsigned long int	4	0 to 4,294,967,295
long long int	8	-(2 ⁶³) to (2 ⁶³)-1
unsigned long long int	8	0 to 18,446,744,073,709,551,615
signed char	1	-128 to 127
unsigned char	1	0 to 255
float	4	
double	8	
long double	12	
wchar_t	2 or 4	1 wide character

Note : Above values may vary from compiler to compiler. In the above example, we have considered GCC 32 bit.

We can display the size of all the data types by using the sizeof() operator and passing the keyword of the datatype as argument to this function as shown below:

```
// C++ program to sizes of data types
#include<iostream>
using namespace std;
int main()
{
    cout << "Size of char : " << sizeof(char)
        << " byte" << endl;
    cout << "Size of int : " << sizeof(int)
        << " bytes" << endl;
    cout << "Size of short int : " << sizeof(short int)
        << " bytes" << endl;
    cout << "Size of long int : " << sizeof(long int)
        << " bytes" << endl;
    cout << "Size of signed long int : " << sizeof(signed long int)
        << " bytes" << endl;
    cout << "Size of unsigned long int : " << sizeof(unsigned long int)
        << " bytes" << endl;
    cout << "Size of float : " << sizeof(float)
        << " bytes" << endl;
    cout << "Size of double : " << sizeof(double)
        << " bytes" << endl;
}
```

```

    cout << "Size of wchar_t : " << sizeof(wchar_t)
        << " bytes" << endl;
    return 0;
}

```

Output:

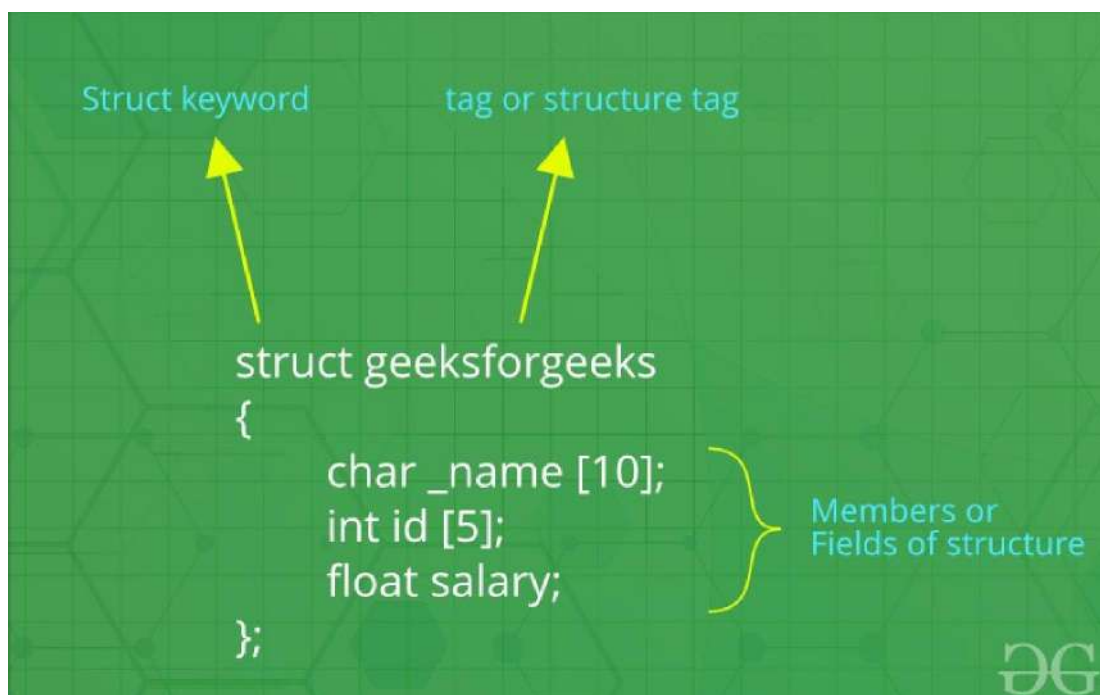
```

Size of char : 1 byte
Size of int : 4 bytes
Size of short int : 2 bytes
Size of long int : 8 bytes
Size of signed long int : 8 bytes
Size of unsigned long int : 8 bytes
Size of float : 4 bytes
Size of double : 8 bytes
Size of wchar_t : 4 bytes

```

What is a structure?

A structure is a user-defined data type in C/C++. A structure creates a data type that can be used to group items of possibly different types into a single type.



Structures in C++

How to create a structure?

The 'struct' keyword is used to create a structure. The general syntax to create a structure is as shown below:

```

struct structureName{
    member1;
    member2;
    member3;
}

```

```

        .
        .

        memberN;
};
Program based on structure data type:-

```

```

#include <iostream>
using namespace std;
struct Point {
    int x, y;
};
int main()
{
    struct Point p1 = { 0, 1 };
    // Accessing members of point p1
    p1.x = 20;
    cout << "x = " << p1.x << ", y = " << p1.y;
    return 0;
}

```

Output

x = 20, y = 1

Enums:

Enums are user-defined types that consist of named integral constants.

- It helps to assign constants to a set of names to make the program easier to read, maintain and understand.
- An Enumeration is declared by using the keyword “**enum**”.

Below is the C++ program illustrating the use of enum:

```

// C++ program to illustrate the use
// of the Enums

#include <bits/stdc++.h>
using namespace std;

// Defining an enum
enum GeeksforGeeks { Geek1,
                    Geek2,
                    Geek3 };

GeeksforGeeks G1 = Geek1;
GeeksforGeeks G2 = Geek2;
GeeksforGeeks G3 = Geek3;

// Driver Code
int main()
{
    cout << "The numerical value "
          << "assigned to Geek1 : "

```

```

        << G1 << endl;

    cout << "The numerical value "
        << "assigned to Geek2 : "
        << G2 << endl;

    cout << "The numerical value "
        << "assigned to Geek3 : "
        << G3 << endl;

    return 0;
}

```

Output

The numerical value assigned to Geek1 : 0

The numerical value assigned to Geek2 : 1

The numerical value assigned to Geek3 : 2

Unions: A union is a type of structure that can be used where the amount of memory used is a key factor.

Unions are declared and used in the same way as structures. Like structures, it is also used to store different types of data.

- It is declared by using the keyword “**union**”.

Defining a Union

Let's look at the syntax of a union.

```

union union_name
{
    data-type member-1;
    data-type member-2;
    data-type member-3;
    data-type member-4;
};

```

Below is the C++ program illustrating the implementation of union:

```

#include <iostream>
using namespace std;
struct student1 {           // defining a struct
    int roll_no;
    char name[40];
    int phone_number;
};
union student2 {           // defining a union
    int roll_no;
    char name[40];
};

```

```

        int phone_number;
};

int main()
{
    struct student1 s1;
    union student2 u1;
    cout << "size of structure : " << sizeof(s1) << endl;
    cout << "size of union : " << sizeof(u1) << endl;
    return 0;
}

```

Output:-

size of structure : 48

size of union : 40

typedef: The typedef is used to give data type a new name. For example,

```
#include <iostream>
```

```
using namespace std;
```

```
typedef char BYTE; // After this line BYTE can be used in place of char
```

```

int main()
{
    BYTE b1, b2;
    b1 = 'c';
    b2 = 'd';
    cout<<b1 <<" " <<b2;
    return 0;
}

```

Friend Keyword:

Friend is a keyword in C++ that is used to share the information of a class that was previously hidden. For example, the **private** members of a class are hidden from every other class. Similarly, the **protected** members are hidden from all classes other than its children classes.

Friend Class A friend class can access private and protected members of other class in which it is declared as friend. It is sometimes useful to allow a particular class to access private members of other class.


```

#include <iostream>
class A {
private:
    int a;

public:
    A() { a = 0; }
    friend class B; // Friend Class
};

class B {
private:
    int b;

public:
    void showA(A& x)
    {
        // Since B is friend of A, it can access
        // private members of A
        std::cout << "A::a=" << x.a;
    }
};

int main()
{
    A a;
    B b;
    b.showA(a);
    return 0;
}

```

Constant Variables(const):

There are a certain set of rules for the declaration and initialization of the constant variables:

- The [const variable](#) cannot be left un-initialized at the time of the assignment.
- It cannot be assigned value anywhere in the program.
- Explicit value needed to be provided to the constant variable at the time of declaration of the constant variable.

- i) const int x; //wrong
- ii) const int x; //wrong
 x=5;
- iii) const int x=5; //True

Auto:

Auto is used for the deduction of data-type of variables and functions at compile time. Therefore, the programmer is not obliged to specify the exact data-types and leave it to the compiler.

```

auto i = 15;
auto it = &i;
cout<<i<<endl;
cout<<*it<<endl;

```

Output:-
15
15

Register:

Registers are faster than memory to access, so the variables which are most frequently used in a C++ program can be put in registers using *register* keyword. The keyword *register* hints to compiler that a given variable can be put in a register.

```
register int x=99;
```

Extern:

External variables are also known as global variables. These variables are defined outside the function and are available globally throughout the function execution. The “extern” keyword is used to declare and define the external variables.

```
extern int x = 32;
```

```
#include <iostream>
using namespace std;
extern int x = 32;
int b = 8;
int main() {
    extern int b;
    cout<<"The value of extern variables x and b " <<x <<" " << b<<"\n";
    x = 15;
    cout<<"The value of extern variables x and b " <<x <<" " << b;
    return 0;
}
```

Output:

The value of extern variables x and b 32 8

The value of extern variables x and b 15 8

Typedef:

The typedef keyword allows the programmer to create new names for types such as int or, more commonly in C++, templated types--it literally stands for "type definition".

The typedef is used to give data type a new name. For example:-

```
typedef char BYTE;
```

static:

When a variable is declared as static, space for **it gets allocated for the lifetime of the program**. Even if the function is called multiple times, space for the static variable is allocated only once and the value of variable in the previous call gets carried through the next function call.

Static variables are initialized only once. Compiler persist the variable till the end of the program. Static variable can be defined inside or outside the function. They are local to the block. The default value of static variable is zero. The static variables are alive till the execution of the program.

```

#include <iostream>
#include <string>
using namespace std;
void demo()
{
    // static variable
    static int count = 0;
    cout << count << " ";
    // value is updated and
    // will be carried to next
    // function calls
    count++;
}
int main()
{
    for (int i=0; i<5; i++)
        demo();
    return 0;
}

```

Output:

0 1 2 3 4

Default:

The **=default** keyword commands the compiler to generate a synthesized version of certain function. For instance, if you do not want to define the default constructor explicitly, we can define the function as '=default'.

```

class A
{
    Public:
    A() =default; //compiler will generate a default constructor.
};

```

Return:

The **return statement** returns the flow of the execution to the [function](#) from where it is called. This statement does not mandatorily need any conditional statements. As soon as the statement is executed, the **flow of the program stops immediately** and returns the control from where it was called. The return statement may or may not return anything for a void function, but for a non-void function, a return value must be returned.

Void:

void is a keyword in c++, It means null. The function which declared as void will not return any values.
It is simply keyword that is used to tell compiler that function will not return any value.

Continue:

The **continue** statement breaks one iteration (in the loop), if a specified condition occurs, and continues with the next iteration in the loop.

```
#include <iostream>
using namespace std;

int main() {
    for (int i = 0; i < 10; i++) {
        if (i == 4) { //when i=4 continue for next iteration without printing.
            continue;
        }
        cout << i << " ";
    }
    return 0;
}
```

Output:- 0 1 2 3 5 6 7 8 9

Manipulators:

Manipulators are helping functions that can modify the [input/output](#) stream.

Manipulators are helper functions that make it possible to control input/output streams using `operator<<` or `operator>>`.

Manipulators are special functions that can be included in the I/O statement to alter the format parameters of a stream.

Types of Manipulators in C++

They are of two types one taking arguments and the other without argument.

1. Non-argument manipulators (Without Parameters)

Non-argument manipulators are also known as "Parameterized manipulators". These manipulators require `iomanip` header. Examples are `setprecision`, `setw` and `setfill`.

2. Argumented manipulators (With parameters)

Argument manipulators are also known as "Non parameterized manipulators". These manipulators require `iostream` header. Examples are `endl`, `fixed`, `showpoint`, `left` and `flush`.

Examples:

- **setw (int n)** – To set field width to n
- **Setbase** – To set the base of the number system
- **endl** – Gives a new line
- **skipws** – Omits white space in input
- **noskipws** – Does not omit white space in the input
- **ends** – Adds null character to close an output string
- **precision (int p)** – The precision is fixed to p

For example,

```
int number = 100;  
  
cout << "Hex Value =" << " " << hex << number << endl;  
cout << "Octal Value=" << " " << oct << number << endl;  
cout << "Setbase Value=" << " " << setbase(16) << number << endl;
```

The output of the above code is:

Hex Value = 0064

Octal Value = 144

Setbase Value= 0064

Unit 2

Content of Unit 2-

Unit 2.0 Functions and Classes	
2.1 Functions in C++	2.4 Array as a class member, Array of objects
2.1.1 Introduction to Functions, function prototyping, Call by Reference	2.5 Passing object as a function argument
2.1.2 Nesting member functions	2.6 Static data member & Member function
2.1.3 Recursive Function, function overloading,	2.7 Access specifiers in C++ and their scope
2.2 Classes and Objects	2.8 C++ streams, C++ stream classes
2.2.1 Introduction to class, Types of class, Base class, Derived class, Virtual class, Abstract class, Friend class	2.9 Introduction to Class template
2.2.2 Specifying Class, defining Data member & member functions	
2.3 Object creation, Memory allocation for objects	

Functions:-

A function is a group of statements that together perform a task. Every C++ program has at least one function, which is **main()**, and all the most trivial programs can define additional functions.

You can divide up your code into separate functions. How you divide up your code among different functions is up to you, but logically the division usually is such that each function performs a specific task.

A function is a set of statements that take inputs, do some specific computation and produces output.

Importance of functions

- Functions help us in reducing code redundancy. If functionality is performed at multiple places in software, then rather than writing the same code, again and again, we create a function and call it everywhere. This also helps in maintenance as we have to change at one place if we make future changes to the functionality.
- Functions make code modular. Consider a big file having many lines of code. It becomes really simple to read and use the code if the code is divided into functions.
- Functions provide abstraction. For example, we can use library functions without worrying about their internal working.

Function

Declaration

A function declaration tells the compiler about the return type of function, name of function, number of parameters function takes and data-types of parameters.

Prototype for function declaration:-

ReturnType FunctionName(parameterLists);

Example: `int max(int num1, int num2);`

Function Definition:

A function definition contains all the details of a function. The details contains function name, return type, parameter lists and most importantly it contains the code of the function.

The statement which will be executed by function when it is called, will be present inside function definition.

```
// function returning the maximum of given two numbers.
int max(int num1, int num2)
{
    // local variable declaration
    int result;
    if (num1 > num2)
        result = num1;
    else
        result = num2;
    return result;
}

// An example function that takes two parameters 'x' and 'y'
// as input and returns max of two input numbers
int max(int x, int y) // x and y are formal parameter.
{
    if (x > y)
        return x;
    else
        return y;
}

// main function that doesn't receive any parameter and
// returns integer.
int main(void)
{
    int a = 10, b = 20;
    // Calling above function to find max of 'a' and 'b'
    int m = max(a, b); // a and b are actual parameter
    printf("m is %d", m);
    return 0;
}
```

Parameter Passing to functions

The parameters passed to function are called **actual parameters**. For example, in the above program 10 and 20 are actual parameters.

The parameters received by function are called **formal parameters**. For example, in the above program x and y are formal parameters.

There are two most popular ways to pass parameters.

Pass by Value:-

In this parameter passing method, values of actual parameters are copied to function's formal parameters and the two types of parameters are stored in different memory locations. So any changes made inside functions are not reflected in actual parameters of caller.

```
#include <iostream>
using namespace std;

void fun(int x) {
    x = 30;
}

int main() {
    int x = 20;
    fun(x); // parameter passed via call by value.
    cout << "x = " << x;
    return 0;
}
```

Pass by Reference:-

Both actual and formal parameters refer to same locations, so any changes made inside the function are actually reflected in actual parameters of caller. Parameters are always passed by value in C. For example. in the below code, value of x is not modified using the function fun().

```
# include <stdio.h>
void fun(int *ptr)
{
    *ptr = 30;
}
int main()
{
    int x = 20;
    fun(&x);
    printf("x = %d", x);
    return 0;
}
```

Nesting of member functions

The public members of a class can be accessed by the object of that class, using dot operator. However a member function can call another member function of the same class directly without using the dot operator. This is called as *nesting of member functions*.

```
#include <iostream>
using namespace std;
class average
{
    int a,b;
public:
    void read();
    void print();
    int avg();
};
void average::read()
{
    cout<<"\n enter a and b: ";
    cin>>a>>b;
}
```



```

void average::print()
{
cout<<"value of a: "<<a;
cout<<"\nvalue of b: "<<b;
cout<<"\naverage is : "<<avg();
}
int average::avg()
{
return (a+b)/2;
}
main()
{
average A;
A.read();
A.print();
}

```

Output:-

Enter a and b: 10 14

Value of a: 10

Value of b: 14

Average is :12

C++ Recursion

When function is called within the same function, it is known as recursion in C++. The function which calls the same function, is known as recursive function.

Syntax:-

```

recursionfunction(){
recursionfunction(); //calling self function
}

```

Program based on recursion-

```

#include<iostream>
using namespace std;
int main()
{
int factorial(int);
int fact,value;
cout<<"Enter any number: ";
cin>>value;
fact=factorial(value);
cout<<"Factorial of a number is: "<<fact<<endl;
return 0;
}

```

```

}
int factorial(int n)
{
if(n<0)
return(-1); /*Wrong value*/
if(n==0)
return(1); /*Terminating condition*/
else
{
return(n*factorial(n-1));
}
}

```

Output:-

Enter any number: 5

Factorial of a number is: 120

We can understand the above program of recursive method call by the figure given below:

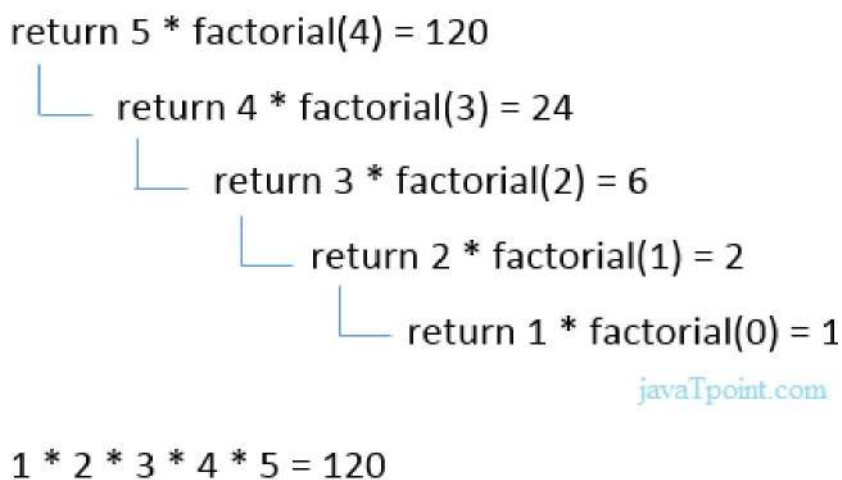


Fig: Recursion

Function Overloading in C++

Function overloading is a feature of object oriented programming where two or more functions can have the same name but different parameters.

When a function name is overloaded with different jobs it is called Function Overloading.

In Function Overloading “Function” name should be the same and the arguments should be different.

Function overloading can be considered as an example of polymorphism feature in C++.

These functions having the same name but different arguments are known as overloaded functions.

For example:

```
// same name different arguments
```

```
int test() { }
```

```
int test(int a) { }
```

```
float test(double a) { }
```

```
int test(int a, double b) { }
```

Following is a simple C++ example to demonstrate function overloading

```
#include <iostream>
using namespace std;

void print(int i) {
    cout << " Here is int " << i << endl;
}

void print(double f) {
    cout << " Here is float " << f << endl;
}

void print(char c) {
    cout << " Here is char " << c << endl;
}

int main() {
    print(10);
    print(10.10);
    print('t');
    return 0;
}
```

Output:

Here is int 10

Here is float 10.1

Here is char t

Class: A class in C++ is the building block that leads to Object-Oriented programming. It is a user-defined data type, which holds its own data members and member functions, which can be accessed and used by creating an instance of that class. A C++ class is like a blueprint for an object.

For Example: Consider the Class of **Cars**. There may be many cars with different names and brand but all of them will share some common properties like all of them will have *4 wheels*, *Speed Limit*, *Mileage range* etc. So here, Car is the class and wheels, speed limits, mileage are their properties.

- A Class is a user defined data-type which has data members and member functions.
- Data members are the data variables and member functions are the functions used to manipulate these variables and together these data

members and member functions defines the properties and behavior of the objects in a Class.

- In the above example of class *Car*, the data member will be *speed limit*, *mileage* etc and member functions can be *apply brakes*, *increase speed* etc.

An **Object** is an instance of a Class. When a class is defined, no memory is allocated but when it is instantiated (i.e. an object is created) memory is allocated.

Defining Class and Declaring Objects

A class is defined in C++ using keyword `class` followed by the name of class. The body of class is defined inside the curly brackets and terminated by a semicolon at the end.

keyword user-defined name

```
class ClassName

{ Access specifier:           //can be private,public or protected

  Data members;              // Variables to be used

  Member Functions() { }     //Methods to access data members

};                            // Class name ends with a semicolon
```

Declaring Objects: When a class is defined, only the specification for the object is defined; no memory or storage is allocated. To use the data and access functions defined in the class, you need to create objects.

Syntax:

`ClassName ObjectName;`

Accessing data members and member functions: The data members and member functions of class can be accessed using the dot('.') operator with the object. For example if the name of object is *obj* and you want to access the member function with the name *printName()* then you will have to write *obj.printName()* .

Accessing Data Members

The public data members are also accessed in the same way given however the private data members are not allowed to be accessed directly by the object. Accessing a data member depends solely on the access control of that data member.

This access control is given by [Access modifiers in C++](#). There are three access modifiers : **public**, **private** and **protected**.

```
// C++ program to demonstrate
// accessing of data members

#include <iostream>
using namespace std;
class Geeks
{
    // Access specifier
    public:

    string geekname; // Data Members
    void printname() // Member Functions()
    {
        cout << "Geekname is: " << geekname;
    }
};

int main() {
    Geeks obj1; // Declare an object of class geeks
    obj1.geekname = "Abhi"; // accessing data member
    obj1.printname(); // accessing member function
    return 0;
}
```

Output:

Geekname is: Abhi

Types of class:-

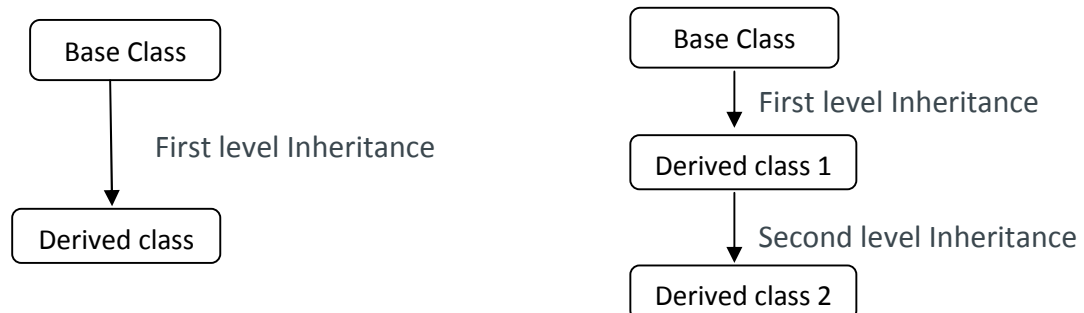
When creating a class, instead of writing completely new data members and member functions, the programmer can designate that the new class should inherit the members of an existing class. This existing class is called the base class, and the new class is referred to as the derived class.

A Base class, as the name suggests, is a class which has well defined data members and functions and acts as a base for another class to derive from.

A Derived class is a class which is derived from the existing base class. It inherits the properties of the base class.

Base Class: A base class is a class in [Object-Oriented Programming language](#), from which other classes are derived. The class which inherits the base class has all members of a base class as well as can also have some additional properties. The Base class members and member functions are inherited to Object of the derived class. A base class is also called **parent class** or **superclass**.

Derived Class: A class that is created from an existing class. The derived class inherits all members and member functions of a base class. The derived class can have more functionality with respect to the Base class and can easily access the Base class. A Derived class is also called a **child class** or **subclass**.



Syntax for creating Derive Class:

```
class BaseClass{
    // members....
    // member function
}
class DerivedClass : public BaseClass{
    // members....
    // member function
}
```

```
}
```

```
#include <iostream>
using namespace std;

// Base Class
class Base {
public:
    int a;
};

// Derived Class
class Derived : public Base {
public:
    int b;
};

// Driver Code
int main()
{
    Derived obj; // Initialize an object of derived class
    obj.b = 3; // Assign value to Derived class variable
    obj.a = 4; // Assign value to Base class variable via derived class
    cout << "Value from derived class: " << obj.b << endl;
    cout << "Value from base class: " << obj.a << endl;
    return 0;
}
```

Output:

Value from derived class: 3

Value from derived class: 4

class Derived : public Base

This means that we have created a derived class from the base class in **public mode**.

Alternatively, we can also derive classes in **protected** or **private** modes.

These 3 keywords (`public`, `protected`, and `private`) are known as **access specifiers** in C++ inheritance.

public, protected and private inheritance in C++

public, **protected**, and **private** inheritance have the following features:

- **public inheritance** makes `public` members of the base class `public` in the derived class, and the `protected` members of the base class remain `protected` in the derived class.
- **protected inheritance** makes the `public` and `protected` members of the base class `protected` in the derived class.
- **private inheritance** makes the `public` and `protected` members of the base class `private` in the derived class.

```
class Base {
    public:
        int x;
    protected:
        int y;
    private:
        int z;
};

class PublicDerived: public Base {
    // x is public
    // y is protected
    // z is not accessible from PublicDerived
};

class ProtectedDerived: protected Base {
    // x is protected
    // y is protected
    // z is not accessible from ProtectedDerived
};

class PrivateDerived: private Base {
    // x is private
    // y is private
    // z is not accessible from PrivateDerived
};
```

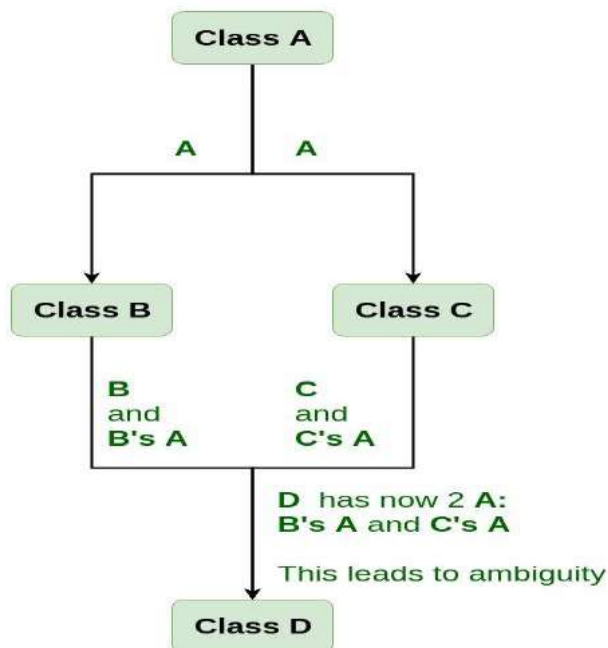
Virtual base class in C++

Virtual base classes are used in virtual inheritance in a way of preventing multiple “instances” of a given class appearing in an inheritance hierarchy when using multiple inheritances.

Need for Virtual Base Classes:

Consider the situation where we have one class **A**. This class is **A** is inherited by two other

classes **B** and **C**. Both these class are inherited into another in a new class **D** as shown in figure below.



As we can see from the figure that data members/function of class **A** are inherited twice to class **D**. One through class **B** and second through class **C**. When any data / function member of class **A** is accessed by an object of class **D**, ambiguity arises as to which data/function member would be called? One inherited through **B** or the other inherited through **C**. This confuses compiler and it displays error.

Example: To show the need of Virtual Base Class in C++

```
#include <iostream>
```

```
using namespace std;
```

```

class A {
public:
    void show()
    {
        cout << "Hello form A \n";
    }
};
class B : public A {};
class C : public A {};
class D : public B, public C {};
int main()
{
    D object;
    object.show();
}
  
```

Output:- Error

To resolve this ambiguity when class **A** is inherited in both class **B** and class **C**, it is declared as **virtual base class** by placing a keyword **virtual** as :

Syntax for Virtual Base Classes:

Syntax 1:

```
class B : virtual public A
{
};
```

Syntax 2:

```
class C : public virtual A
{
};
```

Note: **virtual** can be written before or after the **public**. Now only one copy of data/function member will be copied to class **C** and class **B** and class **A** becomes the virtual base class.

Virtual base classes offer a way to save space and avoid ambiguities in class hierarchies that use multiple inheritances. When a base class is specified as a virtual base, it can act as an indirect base more than once without duplication of its data members. A single copy of its data members is shared by all the base classes that use virtual base.

```
#include <iostream>
using namespace std;
```

```
class A {
public:
    int a;
    A() // constructor
    {
        a = 10;
    }
};
class B : public virtual A {};

class C : public virtual A {};

class D : public B, public C {};
int main()
{
    D object; // object creation of class d
    cout << "a = " << object.a << endl;
    return 0;
}
```

Output:- a = 10

Abstract Class:-

An **abstract class in C++** is a class that has at least *one* pure virtual function (i.e., a function that has no definition). The classes inheriting the abstract class *must* provide a definition for the pure virtual function; otherwise, the subclass would become an abstract class itself.

C++ Pure Virtual Functions:-

Pure virtual functions are used if a function doesn't have any use in the base class but the function must be implemented by all its derived classes. Let's take an example, Suppose, we have derived Triangles, `Square` and `Circle` classes from the `Shape` class, and we want to calculate the area of all these shapes.

In this case, we can create a pure virtual function named `calculateArea()` in the `Shape`. Since it's a pure virtual function, all derived classes `Triangle`, `Square` and `Circle` must include the `calculateArea()` function with implementation.

A pure virtual function doesn't have the function body and it must end with `= 0`. For example,

```
class Shape {
public:
    // creating a pure virtual function
    virtual void calculateArea() = 0;
}
```

The `= 0` syntax doesn't mean we are assigning 0 to the function. It's just the way we define pure virtual functions.

Abstract Class:

A class that contains a pure virtual function is known as an abstract class. In the above example, the class `Shape` is an abstract class.

We cannot create objects of an abstract class. However, we can derive classes from them, and use their data members and member functions (except pure virtual functions).

```
// An abstract class
class Test
{
    // Data members of class
public:
    // Pure Virtual Function
    virtual void show() = 0;

    /* Other members */
}
```

```
#include <iostream>
using namespace std;
// Abstract class shape
class Shape {
public:
    float dimension;
public:
    void getDimension() {
        cin >> dimension;
    }
    // pure virtual Function
    virtual float calculateArea() = 0;
};
```

```
// Derived class
class Square : public Shape {
public:
    float calculateArea() {
        return dimension * dimension;
    }
};

// Derived class
class Circle : public Shape {
public:
    float calculateArea() {
        return 3.14 * dimension * dimension;
    }
};

int main() {
    Square square;
    Circle circle;
    cout << "Enter the length of the square: ";
    square.getDimension();
    cout << "Area of square: " << square.calculateArea() << endl;
    cout << "\nEnter radius of the circle: ";
    circle.getDimension();
    cout << "Area of circle: " << circle.calculateArea() << endl;
    return 0;
}
```

Friend Class A friend class can access private and protected members of other class in which it is declared as friend. It is sometimes useful to allow a particular class to access private members of other class.

```
class MyClass {
private:
    int member1;}
int main() {
    MyClass obj;
    // Error! Cannot access private members from here.
    obj.member1 = 5;}
```

```
class Demo{
private:
int var1=5;
}
Int main()
{
```

```
Demo obj1;  
Cout<<var1;  
return 0;  
}
```

Example:-

```
#include<iostream>  
using namespace std;  
class A  
{  
    int x;  
    public:  
    A()  
    {  
        x=10;  
    }  
    friend class B; //friend class  
};  
  
class B  
{  
    public:  
    void display(A &t)  
    {  
        cout<<endl<<"The value of x="<<t.x;  
    }  
};  
  
main()  
{  
    A _a;  
    B _b;  
    _b.display(_a);  
    return 0;  
}
```

Output

The value of x=10

Arrays within a Class

- Arrays can be declared as the members of a class.
- The arrays can be declared as private, public or protected members of the class.

```
#include<iostream>  
using namespace std;  
const int size=5;  
class student {  
    public:
```

```

int marks[2];

};

int main() {
    student stu;
    stu.marks[0]=10;
    cout<<stu.marks[0]<<" "<<stu.marks[1];
    return 0;
}

```

Array of Objects

When a class is defined, only the specification for the object is defined; no memory or storage is allocated. To use the data and access functions defined in the class, you need to create objects.

The Array of Objects stores *objects*. An array of a class type is also known as an array of objects.

Syntax:-

```
ClassName ObjectName[number of objects];
```

Example#1:

Storing more than one Employee data. Each employee has Employee Id and it's name. Let's assume there is an array of objects for storing employee data emp[4].

Objects	Employee Id	Employee name
emp[0]→		
emp[1]→		
emp[2]→		
emp[3]→		

Program to illustrate array of objects:-

```

#include<iostream>
using namespace std;
class Employee
{
    int id;
public:
    void setId(void)
    { cout << "Enter the id of employee" << endl;
      cin >> id;
    }
}

```

```

    }
    void getId(void)
    {
        cout << "The id of this employee is " << id << endl;
    }
};

int main()
{
    Employee fb[4];
    for (int i = 0; i < 4; i++)
    {
        fb[i].setId();
        fb[i].getId();
    }
    return 0;
}

```

Passing and Returning Objects in C++

In C++ we can pass class's objects as arguments and also return them from a function the same way we pass and return other variables. No special keyword or header file is required to do so.

Passing an Object as argument

To pass an object as an argument we write the object name as the argument while calling the function the same way we do it for other variables.

Syntax:

```
function_name(object_name);
```

```

#include<iostream>
using namespace std;
class complex{
    int a,b;
public:
    void setData(int v1, int v2){
        a = v1;
        b = v2;
    }
    void setDataBySum(complex o1, complex o2){
        a = o1.a + o2.a;
        b = o1.b + o2.b;
    }
    void printNumber(){
        cout<<"Your complex number is "<<a<<" + "<<b<<"i"<<endl;
    }
};

int main(){
    complex c1, c2, c3;
    c1.setData(1, 2);
}

```

```

c1.printNumber();
c2.setData(3, 4);
c2.printNumber();
c3.setDataBySum(c1, c2);
c3.printNumber();
return 0;
}

```

Output:-

Your complex number is 1 + 2i

Your complex number is 3 + 4i

Your complex number is 4 + 6i

Static data members in C++

Static data members are class members that are declared using static keywords. A static member has certain special characteristics. These are:

- Only one copy of that member is created for the entire class and is shared by all the objects of that class, no matter how many objects are created.
- It is initialized before any object of this class is being created, even before main starts.
- It is visible only within the class, but its lifetime is the entire program

Syntax:-

```
static data_type data_member_name;
```

```

#include <iostream>
using namespace std;
class Car
{
private:
int id;
char name[20];
int marks;
public:
static int static_member;
void inp()
{
cout << "\n\n Enter the Id of the Car: " << endl;
cin >> id;
cout << " Enter the name of the Car: " << endl;
cin >> name; static_member++;
}
void disp ()
{
cout << "\n Id of the Car: " << id;

```



```

cout << "\n Name of the Car: " << name;
}
};
int Car::static_member = 0;
int main ()
{
Car c1;
c1. inp ();
c1. disp();
Car c2;
c2. inp ();
c2. disp();
cout << " \n No. of cars created in the class: " << Car::static_member << endl;
return 0;
}

```

Output:-

Enter the Id of the Car: 12

Enter the name of the Car: Tata

Id of the Car: 12

Name of the Car: tata

Enter the Id of the Car: 1

Enter the name of the Car: mercedes

Id of the Car: 1

Name of the Car: mercedes

No. of cars created in the class: 2

Static Member Functions

The static member functions are special functions used to access the static data members or other static member functions. A member function is defined using the static keyword. A static member function shares the single copy of the member function to any number of the class' objects. We can access the static member function using the class name or class' objects. If the static member function accesses any non-static data member or non-static member function, it throws an error.

Syntax

```
class_name::function_name (parameter);
```

Program to Demonstrate static member function:-

```

#include <iostream>
using namespace std;
class Note
{
static int num; // declare a static data member
public:
static int func () // create static member function
{
return num;
}
}

```

```

}
};
// initialize the static data member using the class name and the scope resolution operator
int Note :: num = 5;
int main ()
{
// access static member function using the class name and the scope resolution
cout << " The value of the num is: " << Note:: func () << endl;
return 0;
}

```

Output:-

The value of the num is: 5

Access Modifiers in C++

Access modifiers are used to implement an important aspect of Object-Oriented Programming known as **Data Hiding**. Consider a real-life example: The Research and Analysis Wing (R&AW), having 10 core members, has come into possession of sensitive confidential information regarding national security. Now we can correlate these core members to data members or member functions of a class, which in turn can be correlated to the R&A Wing. These 10 members can directly access the confidential information from their wing (the class), but anyone apart from these 10 members can't access this information directly, i.e., outside functions other than those prevalent in the class itself can't access the information (that is not entitled to them) without having either assigned privileges (such as those possessed by a friend class or an inherited class, as will be seen in this article ahead) or access to one of these 10 members who is allowed direct access to the confidential information (similar to how private members of a class can be accessed in the outside world through public member functions of the class that have direct access to private members). This is what data hiding is in practice.

Access Modifiers or Access Specifiers in a [class](#) are used to assign the accessibility to the class members, i.e., they set some restrictions on the class members so that they can't be directly accessed by the outside functions.

There are 3 types of access modifiers available in C++:

1. **Public**
2. **Private**
3. **Protected**

Note: If we do not specify any access modifiers for the members inside the class, then by default the access modifier for the members will be **Private**.

1. Public: All the class members declared under the public specifier will be available to everyone. The data members and member functions declared as public can be accessed by other classes and functions too. The public

members of a class can be accessed from anywhere in the program using the direct member access operator (.) with the object of that class.

Program to demonstrate public access specifier:-

```
// C++ program to demonstrate public
// access modifier
#include<iostream>
using namespace std;
// class definition
class Circle
{
public:
double radius;
double compute_area()
{
return 3.14*radius*radius;
}
};
// main function
int main()
{
Circle obj;
obj.radius = 5.5; // accessing public datamember outside class
cout << "Radius is: " << obj.radius << "\n";
cout << "Area is: " << obj.compute_area();
return 0;
}
```

Output:-

Radius is: 5.5

Area is: 94.985

In the above program, the data member *radius* is declared as public so it could be accessed outside the class and thus was allowed access from inside main().

2. Private: The class members declared as *private* can be accessed only by the member functions inside the class. They are not allowed to be accessed directly by any object or function outside the class. Only the member functions or the [friend functions](#) are allowed to access the private data members of the class.

```
// C++ program to demonstrate private
// access modifier

#include<iostream>
using namespace std;
class Circle
{
// private data member
private:
double radius;
// public member function
public:
double compute_area()
{ // member function can access private
// data member radius
}
```

```

        return 3.14*radius*radius;
    }
};
// main function
int main()
{
    // creating object of the class
    Circle obj;
    // trying to access private data member
    // directly outside the class
    obj.radius = 1.5;
    cout << "Area is:" << obj.compute_area();
    return 0;
}

```

Output:-

In function 'int main()':

11:16: error: 'double Circle::radius' is private

double radius;

31:9: error: within this context

obj.radius = 1.5;

because we are not
directly from outside
radius being a private

```

// C++ program to demonstrate private
// access modifier
#include<iostream>
using namespace std;
class Circle
{
    // private data member
    private:
        double radius;
    // public member function
    public:
        void compute_area(double r)
        {    // member function can access private
            // data member radius
            radius = r;
            double area = 3.14*radius*radius;
            cout << "Radius is: " << radius << endl;
            cout << "Area is: " << area;
        }
};
// main function
int main()
{
    // creating object of the class
    Circle obj;
    // trying to access private data member
    // indirectly outside the class
    obj.compute_area(1.5);
    return 0;
}

```

Output:-

Radius is: 1.5

Area is: 7.065

3. Protected: The protected access modifier is similar to the private access modifier in the sense that it can't be accessed outside of its class unless with the help of a friend class. The difference is that the class members declared as Protected can be accessed by any subclass (derived class) of that class as well.

```
// C++ program to demonstrate
// protected access modifier
#include <iostream>
using namespace std;
// base class
class Parent
{
    // protected data members
    protected:
    int id_protected;
};

// sub class or derived class from public base class
class Child : public Parent
{
    public:
    void setId(int id)
    {
        // Child class is able to access the inherited
        // protected data members of base class
        id_protected = id;
    }
    void displayId()
    {
        cout << "id_protected is: " << id_protected << endl;
    }
};

// main function
int main() {
    Child obj1;
    // member function of the derived class can
    // access the protected data members of the base class
    obj1.setId(81);
    obj1.displayId();
    return 0;
}
```

Output:-

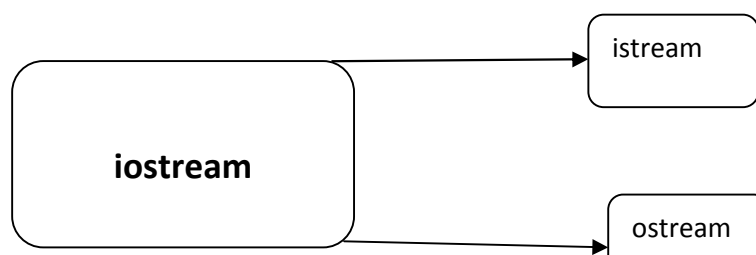
id_protected is: 81

C++ Stream and C++ Stream Classes Structure

A 'stream' is nothing but a series of characters. The characters may be either normal characters (char) or with some special characters (wchar_t).

Stream classes in C++ are used to input and output operations on files and io devices. These classes have specific features and to handle input and output of the program.

The **iostream.h** library holds all the stream classes in the C++ programming language.



ios class – This class is the base class for all stream classes. The streams can be input or output streams. This class defines members that are independent of how the templates of the class are defined.

istream Class – The istream class handles the input stream in c++ programming language. These input stream objects are used to read and interpret the input as a sequence of characters. The cin handles the input.

ostream class – The ostream class handles the output stream in c++ programming language. These output stream objects are used to write data as a sequence of characters on the screen. cout and puts handle the out streams in c++ programming language.

Examples of OUT stream:-

```
Program 1:-
#include <iostream>
using namespace std;
int main(){
    puts("This output is printed using puts");
}
Program 2:-
#include <iostream>
using namespace std;
int main(){
    cout<<"This output is printed on screen";
}
```

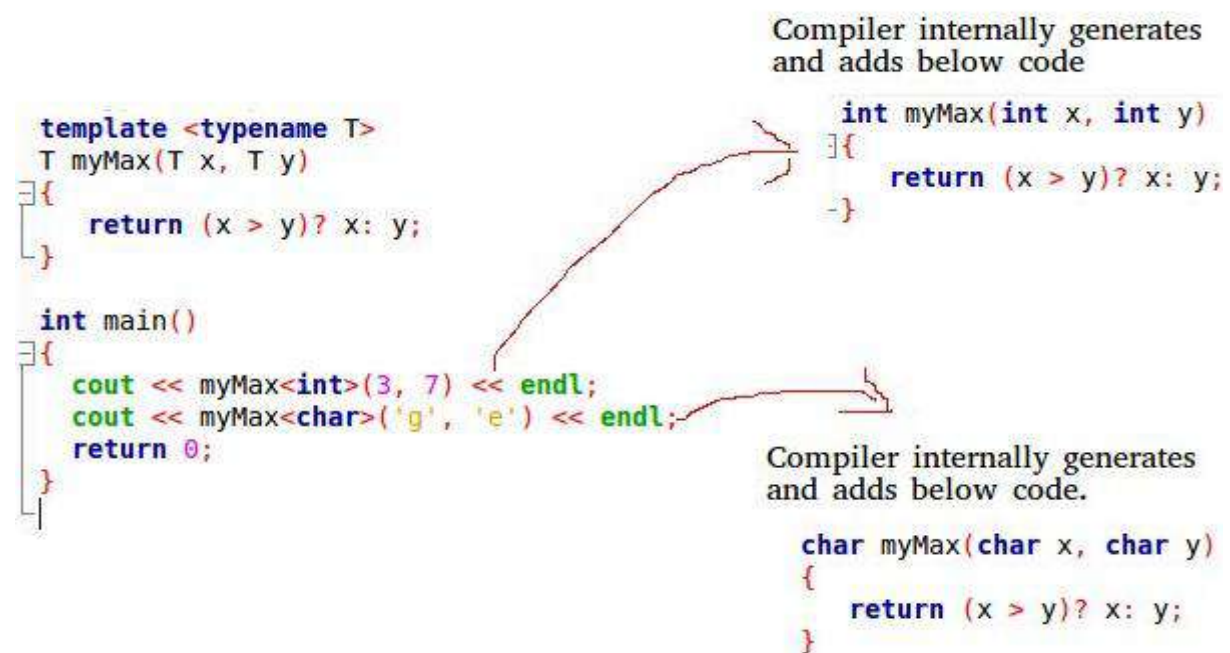
Examples of IN stream:-

```
Program 1:-
#include <iostream>
using namespace std;
int main()
{
    int no;
    cout<<"Enter a number ";
    cin>>no;
    cout<<"Number entered using cin is "<<no;
}
```

```
Program 2:-
#include <iostream>
using namespace std;
int main(){
    char ch[10];
    puts("Enter a character array");
    gets(ch);
    puts("The character array entered using gets is : ");
    puts(ch);
}
```

sort() for different data types. Rather than writing and maintaining the multiple codes, we can write one sort() and pass data type as a parameter. The keyword `template` always begins a definition and declaration of a function template and class template. The `<>` brackets contain a list of template parameters.

The keyword `typename` tells the compiler that what follows is the name of a type. To use the template function you call it with the required type in angle-brackets (`< >`)



```

#include <iostream>
using namespace std;
// One function works for all data types. This would work
// even for user defined types if operator '>' is overloaded
template <typename T> T myMax(T x, T y)
{
    return (x > y) ? x : y;
}
int main()
{
    cout << myMax<int>(3, 7) << endl; // Call myMax for int

    cout << myMax<char>('g', 'e')
        << endl; // call myMax for char
    return 0;
}

```

Output:-

7

g

Unit 3.0 Constructors and Destructors

3.1 Constructors

3.1.1 Concept of Initialization using constructor

3.1.2 Multiple constructors in a class

Dynamic, Constructor with default arguments

3.1.3 Types of constructors, Default, Copy, Parameterized,

3.2 Destructor

Introduction to Constructors:

Constructor in C++ is a special method (function) that is invoked automatically at the time of object creation. It is used to initialize the data members of new objects generally. The constructor in C++ has the same name as the class. Constructor is invoked at the time of object creation. It constructs the values i.e. provides data for the object which is why it is known as constructors.

Constructor does not have a return value, hence they do not have a return type.

The prototype of Constructors is as follows:

```
<class-name> (list-of-parameters);
```

Constructors can be defined inside or outside the class declaration:-

1. The syntax for defining the constructor within the class:-

```
<class-name> (list-of-parameters) { // constructor definition }
```

```
class ABC
```

```
{
```

```
.....
```

```
public :
```

```
    ABC(parameter_list) // constructor definition
```

```
    {
```

```
        .....
```

```
    }
```

```
        .....
```

```
};
```

2. The syntax for defining the constructor outside the class:-

```
<class-name>::<class-name> (list-of-parameters)
```

```
{ // constructor definition}
```

```
class XYZ
```

```
{
```

```
.....
```

```
public:
```

```
    XYZ (parameter_list);//Constructor declaration
```

```
.....
```

```
};
```

```
XYZ::XYZ(parameter_list)//Constructor Definition
```

```
{
```

```
.....
```

```
}
```

A constructor is different from normal functions in following ways:

- Constructor has same name as the class itself
- Default Constructors don't have input argument however, Copy and Parameterized Constructors have input arguments
- Constructors don't have return type
- A constructor is automatically called when an object is created.
- It must be placed in public section of class.
- If we do not specify a constructor, C++ compiler generates a default constructor for object (expects no parameters and has an empty body).

Initialization using constructor:-

//A simple Constructor program:-

```
#include <iostream>
```

```
using namespace std;
```

```
class Employee
```

```
{
```

```
public:
```

```
    Employee()
```

```
{
```

```
    cout<<"Default Constructor Invoked"<<endl;
```

```

    }
};
int main(void)
{
    Employee e1; //creating an object of Employee
    Employee e2;
    return 0;
}

```

Output:-

```

Default Constructor Invoked
Default Constructor Invoked
//program to demonstrate initialization using constructor.
#include <iostream>
using namespace std;
class construct
{
    int x,y;
    public:
        construct()
        {
            x=10;
            y=20;
            cout<<"Default Constructor Invoked"<<endl;
            cout<<"The values of x and y are:-"<<x <<" "<<y;
        }
};
int main()
{
    construct c1; //creating an object of construct
    return 0;
}

```

C++ Constructor Overloading (multiple constructor in a class)

In C++, We can have more than one constructor in a class with same name, as long as each has a different list of arguments. This concept is known as Constructor Overloading and is quite similar to function overloading.

Overloaded constructors have the same name (name of the class) but the different number of arguments. Depending upon the number and type of arguments passed, the corresponding constructor is called.

- Overloaded constructors essentially have the same name (exact name of the class) and differ by number and type of arguments.

- A constructor is called depending upon the number and type of arguments passed.
- While creating the object, arguments must be passed to let compiler know, which constructor needs to be called.

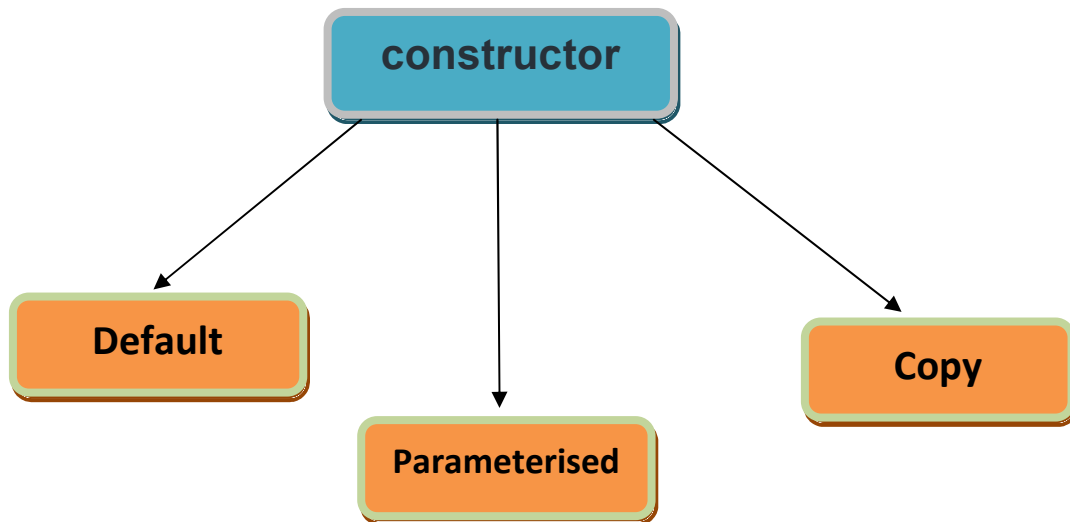
//constructor overloading

```
#include <iostream>
using namespace std;
class construct
{
int x,y;
public:
    construct()
    {
        x=10;
        y=20;
        cout<<"Default Constructor Invoked"<<endl;
        cout<<"The values of x and y are:-"<<x <<" "<<y;
    }
    construct(int a, int b)
    {
        x=a;
        y=b;
        cout<<"Default Constructor Invoked"<<endl;
        cout<<"The values of x and y are:-"<<x <<" "<<y;
    }
};
int main()
{
    construct c1; //creating an object of construct
    construct c2(5,6);
    return 0;
}
```

Output:-

```
Default Constructor Invoked
The values of x and y are:-10 20
Default Constructor Invoked
The values of x and y are:-5 6
```

Types of constructor:-



Let us understand the types of constructors in C++ by taking a real-world example . Suppose you went to a shop to buy a marker. When you want to buy a marker, what are the options. The first one you go to a shop and say give me a marker. So just saying give me a marker mean that you did not set which brand name and which color, you didn't mention anything just say you want a marker. So when we said just I want a marker so whatever the frequently sold marker is there in the market or in his shop he will simply hand over that. And this is what a default constructor is! The second method is you go to a shop and say I want a marker a red in color and XYZ brand. So you are mentioning this and he will give you that marker. So in this case you have given the parameters. And this is what a parameterized constructor is! Then the third one you go to a shop and say I want a marker like this(a physical marker on your hand). So the shopkeeper will see that marker. Okay, and he will give a new marker for you. So copy of that marker. And that's what a copy constructor is!

Types of Constructors

1. Default Constructors: Default constructor is the constructor which doesn't take any argument. It has no parameters. It is also called a zero-argument constructor.

```
// Cpp program to illustrate the
```

```
// concept of Constructors
```

```
#include <iostream>
```

```
using namespace std;
```

```

class construct {
public:
    int a, b;
    // Default Constructor
    construct()
    {
        a = 10;
        b = 20;
    }
};
int main()
{
    // Default constructor called automatically
    // when the object is created
    construct c;
    cout << "a: " << c.a << endl << "b: " << c.b;
    return 1;
}

```

Output

a: 10

b: 20

Note: Even if we do not define any constructor explicitly, the compiler will automatically provide a default constructor implicitly.

```

// C++ program to illustrate
// Constructor overloading
#include <iostream>
using namespace std;

class construct
{

public:
    float area;

    // Constructor with no parameters
    construct()
    {
        area = 0;
    }

    // Constructor with two parameters

```

```

    construct(int a, int b)
    {
        area = a * b;
    }

    void disp()
    {
        cout<< area<< endl;
    }
};

int main()
{
    // Constructor Overloading
    // with two different constructors
    // of class name
    construct o;
    construct o2( 10, 20);

    o.disp();
    o2.disp();
    return 1;
}

// Example
#include<iostream>
using namespace std;
class student
{
    int rno;
    char name[50];
    double fee;
public:
    student()          // Explicit Default constructor
    {
        cout<<"Enter the RollNo:";
        cin>>rno;
        cout<<"Enter the Name:";
        cin>>name;
        cout<<"Enter the Fee:";
        cin>>fee;
    }
}

```

```

void display()
{
    cout<<endl<<no<<"\t"<<name<<"\t"<<fee;
}
};

int main()
{
    student s;
    s.display();
    return 0;
}

```

2. Parameterized Constructors: It is possible to pass arguments to constructors. Typically, these arguments help initialize an object when it is created. To create a parameterized constructor, simply add parameters to it the way you would to any other function. When you define the constructor's body, use the parameters to initialize the object.

Note: when the parameterized constructor is defined and no default constructor is defined explicitly, the compiler will not implicitly call the default constructor and hence creating a simple object will throw an error.

For example:- In this program there is no default constructor and we are creating a simple object so it throws an error-

```

#include <iostream>
using namespace std;
class construct
{
    int x,y;
public:
    construct(int a, int b)
    {
        x=a;
        y=b;
        cout<<"Default Constructor Invoked"<<endl;
        cout<<"The values of x and y are:-"<<x <<" "<<y;
    }
};

```



```

int main()
{
construct c1; //creating an object of construct
construct c2(5,6);
return 0;
}

```

An example of parameterised constructor

```

// CPP program to illustrate
// parameterized constructors
#include <iostream>
using namespace std;

class Point {
private:
    int x, y;

public:
    // Parameterized Constructor
    Point(int x1, int y1)
    {
        x = x1;
        y = y1;
    }

    int getX() { return x; }
    int getY() { return y; }
};

```

```

int main()
{
    // Constructor called
    Point p1(10, 15);

    // Access values assigned by constructor
    cout << "p1.x = " << p1.getX()
        << ", p1.y = " << p1.getY();

    return 0;
}

```

Uses of Parameterized constructor:

1. It is used to initialize the various data elements of different objects with different values when they are created.

2. It is used to overload constructors.

Copy constructor :

A **copy constructor** is a member function that initializes an object using another object of the same class. A copy constructor has the following general function prototype:

```
ClassName (const ClassName &old_obj);
```

- Copy constructor is used to initialize the members of a newly created object by copying the members of an already existing object.
- Copy constructor takes a reference to an object of the same class as an argument.

```
Sample(Sample &t)
{
    id=t.id;
}
```

- The process of initializing members of an object through a copy constructor is known as copy initialization.
- It is also called member-wise initialization because the copy constructor initializes one object with the existing object, both belonging to the same class on a member by member copy basis.
- The copy constructor can be defined explicitly by the programmer. If the programmer does not define the copy constructor, the compiler does it for us.

// C++ program to demonstrate the working

// of a COPY CONSTRUCTOR

```
#include <iostream>
```

```
using namespace std;
```

```
class Point {
```

```
private:
```

```
    int x, y;
```

```
public:
```

```
    Point(int x1, int y1)
```

```
{
```

```
    x = x1;
```

```
    y = y1;
```

```
}
```

```
// Copy constructor
```

```

Point(const Point& p1)
{
    x = p1.x;
    y = p1.y;
}
int getX() { return x; }
int getY() { return y; }
};

int main()
{
    Point p1(10, 15); // Normal constructor is called here
    Point p2 = p1; // Copy constructor is called here
    // Let us access values assigned by constructors
    cout << "p1.x = " << p1.getX()
        << ", p1.y = " << p1.getY();
    cout << "\np2.x = " << p2.getX()
        << ", p2.y = " << p2.getY();
    return 0;
}

```

Dynamic Constructor in C++

When allocation of memory is done dynamically using dynamic memory allocator new in a constructor, it is known as **dynamic constructor**.

By using this, we can dynamically initialize the objects.

```

#include <iostream>
using namespace std;
class geeks {
    const char* p;
public:
    // default constructor
    geeks()
    {
        // allocating memory at run time
        p = new char[6];
        p = "geeks";
    }
}

```

```

    void display()
    {
        cout << p << endl;
    }
};

int main()
{
    geeks obj;
    obj.display();
}

```

Destructors in C++

Destructor is an instance member function which is invoked automatically whenever an object is going to be destroyed. Meaning, a destructor is the last function that is going to be called before an object is destroyed.

- Destructor is also a special member function like constructor. Destructor destroys the class objects created by constructor.
- Destructor has the same name as their class name preceded by a tiled (~) symbol.
- It is not possible to define more than one destructor.
- The destructor is only one way to destroy the object create by constructor. Hence destructor can-not be overloaded.
- Destructor neither requires any argument nor returns any value.
- It is automatically called when object goes out of scope.
- Destructor release memory space occupied by the objects created by constructor.
- In destructor, objects are destroyed in the reverse of an object creation.

Note:- The thing is to be noted here, if the object is created by using new or the constructor uses new to allocate memory which resides in the heap memory or the free store, the destructor should use delete to free the memory.

Syntax:

Syntax for defining the destructor within the class

```

~ <class-name>()

{

}

```

Syntax for defining the destructor outside the class

```
<class-name>: ~ <class-name>()
```

```
{
```

```
}
```

// Example:

```
#include<iostream>
```

```
using namespace std;
```

```
class Test
```

```
{
```

```
public:
```

```
    Test()
```

```
    {
```

```
        cout<<"\n Constructor executed";
```

```
    }
```

```
    ~Test()
```

```
    {
```

```
        cout<<"\n Destructor executed";
```

```
    }
```

```
};
```

```
main()
```

```
{
```

```
    Test t1,t2;
```

```
    return 0;
```

```
}
```

Unit-4 Operator overloading and Inheritance

**4.0 Operator
Overloading &
Inheritance**

4.1 Defining Operator
overloading

4.2 Rules for operator
overloading

4.3 Overloading Unary
operator using
member function
& friend Function

4.4 Overloading Binary
operator using
member function
& friend Function

4.5 Type Conversions

4.6 Inheritance

4.6.1 Defining
Derived
classes

4.6.2 Types of
inheritance,
Single,
Multiple,

Multilevel,
Hierarchical,
Hybrid

4.6.3 Making private
member
inheritable

4.6.4 Constructors in
derived classes

C++ Overloading (Function and Operator)

If we create two or more members having the same name but different in number or type of parameter, it is known as C++ overloading. Overloading is a compile-time polymorphism (providing different meanings to same function/operator) In C++, we can overload:

- methods,
- constructors, and
- Operators

C++ Operators Overloading

Operator overloading is a compile-time polymorphism in which the operator is overloaded to provide the special meaning to the user-defined data type. Operator overloading is used to overload or redefines most of the operators available in C++. It is used to perform the operation on the user-defined data type. For example, C++ provides the ability to add the variables of the user-defined data type that is applied to the built-in data types. The advantage of Operators overloading is to perform different operations on the same operand.

Operator that cannot be overloaded are as follows:

- Scope operator (::)
- Sizeof
- member selector(.)
- member pointer selector(*)
- ternary operator(?:)

Syntax for C++ Operator Overloading

To overload an operator, we use a special `operator` function. We define the function inside the class or structure whose objects/variables we want the overloaded operator to work with.

```

class className {
    ... ..
    public
        returnType operator symbol (arguments) {
            ... ..
        }
    ... ..
};

```

Rules for the operator overloading:-

1. Only built-in operators like (+, -, *, /, etc) can be overloaded.
2. We cannot overload all of those operators that are not a part of C++ language like '\$'.
3. We can't change the arity of the operators. The arity of an operator is the number of operands that the operator takes.
4. We can overload the unary operator as an only unary operator, and we cannot overload it as a binary operator and similarly, We can overload binary operators as an only binary operator, and we cannot overload it as a unary operator.
5. During the operator overloading, we cannot change the actual meaning of an operator. For example, We cannot overload the plus(+) operator to subtract one value from the other value.
6. The precedence of the operators remains the same during operator overloading.
7. The operator overloading is not possible for built-in data types. At least one user-defined data types must be there.
8. Some operators like assignment "=", address "&" and comma " ," are by default overloaded.
9. When using binary operators overloaded through a member function, the left-hand operand must be an object of the relevant class.

Unary operator overloading:-

The unary operators operate on a single operand and following are the examples of Unary operators –

- The increment (++) and decrement (--) operators.
- The unary minus (-) operator.
- The logical not (!) operator.

The unary operators operate on the object for which they were called and normally, this operator appears on the left side of the object, as in !obj, -obj, and ++obj but sometime they can be used as postfix as well like obj++ or obj--

//Unary operator overloading using member function

```
#include <iostream>
```

```
using namespace std;
```

```
class Distance {
```

```
private:
```

```
    int feet;          // 0 to infinite
```

```
    int inches;        // 0 to 12
```

```
public:
```

```
    // required constructors
```

```
    Distance() {
```

```
        feet = 0;
```

```
        inches = 0;
```

```
    }
```

```
    Distance(int f, int i) {
```

```
        feet = f;
```

```
        inches = i;
```

```
    }
```

```
    // method to display distance
```

```
    void displayDistance() {
```

```
        cout << "F: " << feet << " I:" << inches << endl;
```

```
    }
```

```
    // overloaded minus (-) operator
```

```
    Distance operator- () {
```

```
        feet = -feet;
```

```
        inches = -inches;
```

```
        return Distance(feet, inches);
```

```
    }
```

```
};
```

```
int main() {
```

```
    Distance D1(11, 10), D2(-5, 11);
```

```

-D1;           // apply negation
D1.displayDistance(); // display D1
-D2;           // apply negation
D2.displayDistance(); // display D2
return 0;
}

```

C++ Friend function

If a function is defined as a friend function in C++, then the protected and private data of a class can be accessed using the function.

By using the keyword friend compiler knows the given function is a friend function.

For accessing the data, the declaration of a friend function should be done inside the body of a class starting with the keyword friend.

```

#include<iostream>
using namespace std;
class Box
{
    private:
        int length;
    public:
        Box(int x)
        {
            length=x;
        }
        friend int printLength(Box); //friend function
};
int printLength(Box b)
{
    b.length += 10;
    return b.length;
}
int main()
{
    Box b(10);
    cout<<"Length of box: "<< printLength(b)<<endl;
    return 0;
}

```

```

//Unary operator overloading using friend function
#include<iostream>
using namespace std;
class UnaryFriend
{
    int a=10;
    int b=20;
    int c=30;
public:
    void getvalues()
    {
        cout<<"Values of A, B & C\n";
        cout<<a<<"\n"<<b<<"\n"<<c<<"\n"<<endl;
    }
    void show()
    {
        cout<<a<<"\n"<<b<<"\n"<<c<<"\n"<<endl;
    }
    void friend operator-(UnaryFriend &x);    //Pass by reference
};
void operator-(UnaryFriend &x)
{
    x.a = -x.a;    //Object name must be used as it is a friend function
    x.b = -x.b;
    x.c = -x.c;
}
int main()
{
    UnaryFriend x1;
    x1.getvalues();
    cout<<"Before Overloading\n";
    x1.show();
    cout<<"After Overloading \n";
    -x1;
    x1.show();
    return 0;
}

```

Binary Operator Overloading in C++

An operator which contains two operands to perform a mathematical operation is called the Binary Operator.

Operator overloading is a compile-time polymorphism in which the operator is overloaded to provide the special meaning to the user-defined data type.

Syntax of the Binary Operator Overloading

Following is the Binary Operator Overloading syntax in the C++ Programming language.

- return_type :: operator binary_operator_symbol (arg)
- {
- // function definition
- }

//Binary operator overloading within member function

```
#include <iostream>
```

```
using namespace std;
```

```
class Complex_num
```

```
{
```

```
    int x, y;
```

```
    public:
```

```
        void inp()
```

```
        {
```

```
            cout << " Input two complex number: " << endl;
```

```
            cin >> x >> y;
```

```
        }
```

```
        Complex_num operator + (Complex_num obj)
```

```
        {
```

```
            Complex_num A; // create an object
```

```
            A.x = x + obj.x; // assign values to object
```

```
            A.y = y + obj.y;
```

```
            return (A);
```

```
        }
```

```
        void print() // display the result of addition
```

```
        {
```

```
            cout << x << " + " << y << "i" << "\n";
```

```
        }
```

```
};
```

```
int main ()
```

```
{
```

```
    Complex_num x1, y1, sum; // here we created object of class Addition i.e x1 and y1
```

```
    x1.inp(); // accepting the values
```

```
    y1.inp();
```

```

    sum = x1 + y1; // actual call is like sum=x1.add(y1)
    cout << "\n Entered values are: \n";    // display user entered values
    x1.print();
    y1.print();
    cout << "\n The addition of two complex (real and imaginary) numbers: ";
    sum.print(); // call print function to display the result of addition
    return 0;
}

```

//Binary operator overloading within member function

```
#include <iostream>
```

```
using namespace std;
```

```
class Complex
```

```
{
```

```
    private:
```

```
        int real;
```

```
        int img;
```

```
    public:
```

```
        Complex (int r = 0, int i = 0)
```

```
        {
```

```
            real = r;
```

```
            img = i;
```

```
        }
```

```
        void Display ()
```

```
        {
```

```
            cout << real << "+i" << img;
```

```
        }
```

```
        friend Complex operator + (Complex c1, Complex c2);
```

```
};
```

```
Complex operator + (Complex c1, Complex c2)
```

```
{
```

```
    Complex temp;
```

```
    temp.real = c1.real + c2.real;
```

```
    temp.img = c1.img + c2.img;
```

```
    return temp;
```

```
}
```

```
int main ()
```

```
{
```

```
    Complex C1(5, 3), C2(10, 5), C3;
```

```
    C1.Display();
```

```
    cout << " + ";
```

```

    C2.Display();
    cout << " = ";
    C3 = C1 + C2;
    C3.Display();
}

```

C++ Type Conversion

C++ allows us to convert data of one type to that of another. This is known as type conversion.

There are two types of type conversion in C++.

- I. Implicit Conversion
- II. Explicit Conversion (also known as Type Casting)

Implicit Type Conversion

The type conversion that is done automatically done by the compiler is known as implicit type conversion. This type of conversion is also known as automatic conversion.

```

// Working of implicit type-conversion
#include <iostream>
using namespace std;
int main() {
    // assigning an int value to num_int
    int num_int = 9;
    // declaring a double type variable
    double num_double;
    // implicit conversion
    // assigning int value to a double variable
    num_double = num_int;
    cout << "num_int = " << num_int << endl;
    cout << "num_double = " << num_double << endl;
    return 0;
}

```

//Automatic Conversion from double to int

```

//Working of Implicit type-conversion
#include <iostream>

```

```

using namespace std;

int main() {
    int num_int;
    double num_double = 9.99;

    // implicit conversion
    // assigning a double value to an int variable
    num_int = num_double;

    cout << "num_int = " << num_int << endl;
    cout << "num_double = " << num_double << endl;

    return 0;
}

```

C++ Explicit Conversion

When the user manually changes data from one type to another, this is known as **explicit conversion**. This type of conversion is also known as **type casting**.

```

//C++ Explicit conversion
#include <iostream>
using namespace std;

int main() {
    // initializing a double variable
    double num_double = 3.56;
    cout << "num_double = " << num_double << endl;

    // C-style conversion from double to int
    int num_int1 = (int)num_double;
    cout << "num_int1 = " << num_int1 << endl;

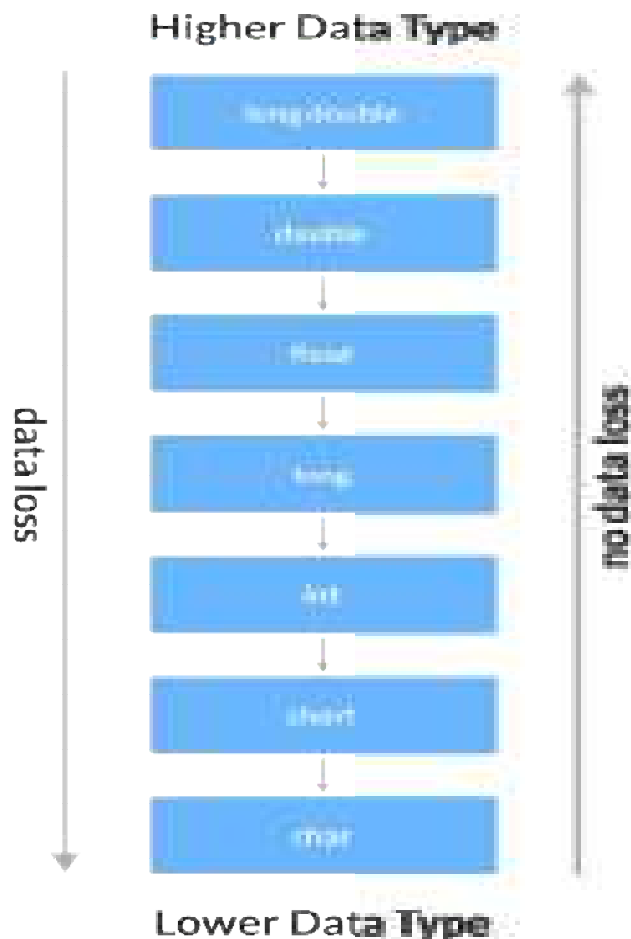
    // function-style conversion from double to int
    int num_int2 = int(num_double);
    cout << "num_int2 = " << num_int2 << endl;

    return 0;
}

```

Data Loss During Conversion (Narrowing Conversion)

As we have seen from the above example, conversion from one data type to another is prone to data loss. This happens when data of a larger type is converted to data of a smaller type.



C++ Inheritance

In C++, inheritance is a process in which one object acquires all the properties and behaviors of its parent object automatically. In such way, you can reuse, extend or modify the attributes and behaviors which are defined in other class.

In C++, the class which inherits the members of another class is called derived class and the class whose members are inherited is called base class. The derived class is the specialized class for the base class.

Types Of Inheritance

C++ supports five types of inheritance:

- Single inheritance
- Multiple inheritance
- Hierarchical inheritance
- Multilevel inheritance
- Hybrid inheritance

The Syntax of Derived class:

```
class derived_class_name : visibility-mode base_class_name
{
    // body of the derived class.
}
```

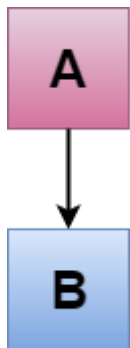
derived_class_name: It is the name of the derived class.

visibility mode: The visibility mode specifies whether the features of the base class are publicly inherited or privately inherited. It can be public or private.

base_class_name: It is the name of the base class.

C++ Single Inheritance

Single inheritance is defined as the inheritance in which a derived class is inherited from the only one base class.



Where 'A' is the base class, and 'B' is the derived class.

//Simple program to demonstrate single level inheritance

```
#include <iostream>
```

```
using namespace std;
```

```
class Account {
```

```
    public:
```

```
    float salary = 60000;
```

```
};
```

```
class Programmer: public Account {
```

```
    public:
```

```

float bonus = 5000;
};
int main(void) {
    Programmer p1;
    cout<<"Salary: "<<p1.salary<<endl;
    cout<<"Bonus: "<<p1.bonus<<endl;
    return 0;
}

```

C++ Multilevel Inheritance

Multilevel inheritance is a process of deriving a class from another derived class.



C++ Multi Level Inheritance Example

When one class inherits another class which is further inherited by another class, it is known as multi level inheritance in C++. Inheritance is transitive so the last derived class acquires all the members of all its base classes.

```

#include <iostream>
using namespace std;
class Animal {
public:
    void eat() {
        cout<<"Eating..."<<endl;
    }
};
class Dog: public Animal

```

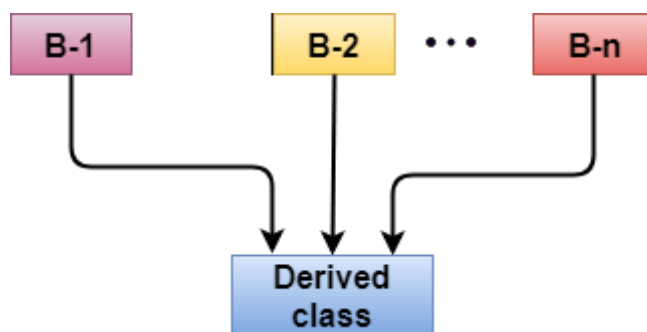
```

{
public:
void bark(){
cout<<"Barking..."<<endl;
}
};
class BabyDog: public Dog
{
public:
void weep() {
cout<<"Weeping...";
}
};
int main(void) {
BabyDog d1;
d1.eat();
d1.bark();
d1.weep();
return 0;
}

```

C++ Multiple Inheritance

Multiple inheritance is the process of deriving a new class that inherits the attributes from two or more classes.



Syntax of the Derived class:

```

class Derived_class : visibility B-1, visibility B-2,.. visibility B-n
{
    // Body of the class;
}

// Program to demonstrate Multiple inheritance

```

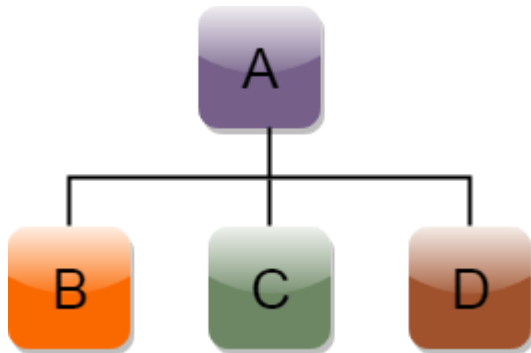
```

#include <iostream>
using namespace std;
class A
{
    protected:
        int a;
    public:
        void get_a(int n)
        {
            a = n;
        }
};
class B
{
    protected:
        int b;
    public:
        void get_b(int n)
        {
            b = n;
        }
};
class C : public A,public B
{
    public:
        void display()
        {
            std::cout << "The value of a is : " <<a<< std::endl;
            std::cout << "The value of b is : " <<b<< std::endl;
            cout<<"Addition of a and b is : "<<a+b;
        }
};
int main()
{
    C c;
    c.get_a(10);
    c.get_b(20);
    c.display();
    return 0;
}

```

C++ Hierarchical Inheritance

Hierarchical inheritance is defined as the process of deriving more than one class from a base class.



Syntax of Hierarchical inheritance:

```
class A
{
    // body of the class A.
}
class B : public A
{
    // body of class B.
}
class C : public A
{
    // body of class C.
}
class D : public A
{
    // body of class D.
}
```

```
#include <iostream>
using namespace std;
class Shape // Declaration of base class.
{
public:
    int a;
    int b;
    void get_data(int n,int m)
    {
        a= n;
```

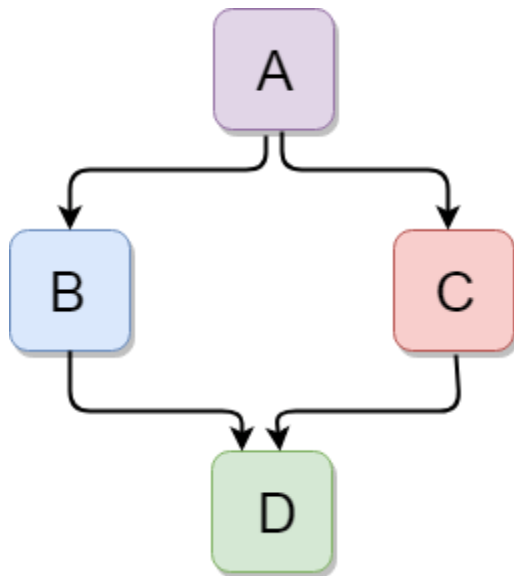
```

        b = m;
    }
};
class Rectangle : public Shape // inheriting Shape class
{
    public:
    int rect_area()
    {
        int result = a*b;
        return result;
    }
};
class Triangle : public Shape // inheriting Shape class
{
    public:
    int triangle_area()
    {
        float result = 0.5*a*b;
        return result;
    }
};
int main()
{
    Rectangle r;
    Triangle t;
    int length,breadth,base,height;
    cout << "Enter the length and breadth of a rectangle: " << endl;
    cin>>length>>breadth;
    r.get_data(length,breadth);
    int m = r.rect_area();
    cout << "Area of the rectangle is : " <<m<< endl;
    cout << "Enter the base and height of the triangle: " << endl;
    cin>>base>>height;
    t.get_data(base,height);
    float n = t.triangle_area();
    cout <<"Area of the triangle is : " << n<< endl;
    return 0;
}

```

C++ Hybrid Inheritance

Hybrid inheritance is a combination of more than one type of inheritance.



```
#include <iostream>
using namespace std;
class A
{
    protected:
    int a;
    public:
    void get_a()
    {
        std::cout << "Enter the value of 'a' : " << std::endl;
        cin >> a;
    }
};

class B : public A
{
    protected:
    int b;
    public:
    void get_b()
    {
        std::cout << "Enter the value of 'b' : " << std::endl;
        cin >> b;
    }
}
```

```

};
class C
{
    protected:
    int c;
    public:
    void get_c()
    {
        std::cout << "Enter the value of c is : " << std::endl;
        cin>>c;
    }
};

class D : public B, public C
{
    protected:
    int d;
    public:
    void mul()
    {
        get_a();
        get_b();
        get_c();
        std::cout << "Multiplication of a,b,c is : " <<a*b*c<< std::endl;
    }
};

int main()
{
    D d;
    d.mul();
    return 0;
}

```

Access Control and Inheritance

A derived class can access all the non-private members of its base class. Thus base-class members that should not be accessible to the member functions of derived classes should be declared private in the base class.

We can summarize the different access types according to - who can access them in the following way -

Access	public	protected	private
--------	--------	-----------	---------

Same class	yes	yes	yes
Derived classes	yes	yes	no
Outside classes	yes	no	no

Accessing private member of a class:-

```
#include <iostream>
using namespace std;
class Engine
{
public:
    Engine(int nc){
        cylinder = nc;
    }
    void start() {
        cout << getCylinder() <<" cylinder engine started" << endl;
    };
    int getCylinder() {
        return cylinder;
    }
private:
    int cylinder;
};
class Car : private Engine
{ // Car has-a Engine
public:
    Car(int nc = 4) : Engine(nc) //constructor of child class calling constructor of base class.
    {}
    void start() {
        cout << "car with " << Engine::getCylinder() <<" cylinder engine started" << endl;
        Engine:: start();
    }
};
int main( )
{
    Car c(8);
    c.start();
    return 0;
}
```

Constructor in derived class

In inheritance, When an object of derived class is created then constructor of derived class get executed and then it calls the constructor of base class.

Syntax for constructor in derived class:-

```
class A
{
public:
    A() {
        //constructor body
    }
};

class B: public A {
public:
    B(): A() //constructor of child class calling constructor of base class.
{
    //constructor body
}
};
```

```
#include <iostream>
using namespace std;
class A
{
    int a;
public:
    A (int k) //parameterized constructor of parent class.
    {
        a = k;
        cout<<"a="<<a<<endl;
    }
};

class B: public A
{
    int b;

public:
    B(int x, int y): A(x)
    {
        b = y;
        cout<<"b="<<b<<endl;
    }
};
```

```

int main()
{
    B obj(2,3);
    return 0;
}

```

Unit-5 Polymorphism and Pointers

Laboratory Instruction (LI)	Class room Instruction (CI)
LE5.1 Debug a program to understand the flow of program execution. Check the values of operands at run time LE5.2 Develop a program for	5.0 Polymorphism and Pointers 5.1 Introduction to Polymorphism, types of polymorphism 5.2 Pointers in C++ 5.2.1 pointer to objects 5.2.2 'this' pointer, pointer to derived classes 5.3 Virtual function, pure virtual function, virtual

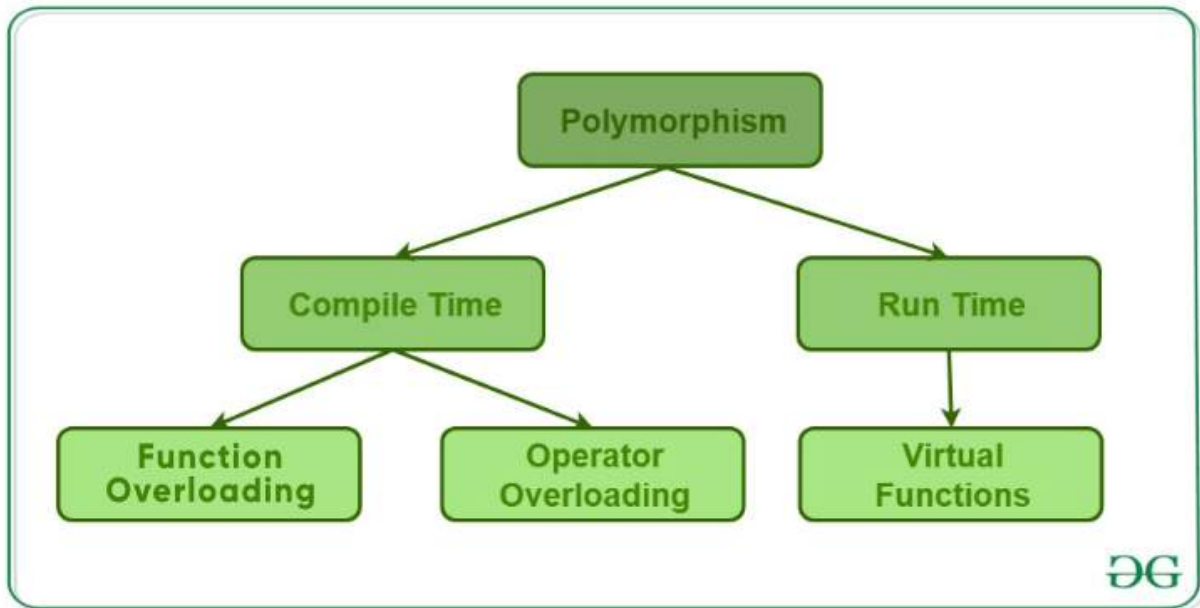
	each type of argument passing technique (call by reference, call by value, call by address)	constructor and destructor
LE5.3	Develop a program to overload common operators like +, -, *, / etc.	

Polymorphism in C++

The word “polymorphism” means having many forms. In simple words, we can define polymorphism as the ability of a message to be displayed in more than one form. A real-life example of polymorphism is a person who at the same time can have different characteristics. Like a man at the same time is a father, a husband and an employee. So the same person exhibits different behavior in different situations. This is called polymorphism. Polymorphism is considered as one of the important features of Object-Oriented Programming. **In C++, polymorphism is mainly divided into two types:**

- Compile-time Polymorphism

- Runtime Polymorphism



1. **Compile-time polymorphism:** This type of polymorphism is achieved by function overloading or operator overloading.

Function Overloading: When there are multiple functions with the same name but different parameters, then the functions are said to be **overloaded**. Functions can be overloaded by **changing the number of arguments** or/and **changing the type of arguments**.

Operator Overloading: C++ also provides the option to overload operators. For example, we can make use of the addition operator (+) for string class to concatenate two strings. We know that the task of this operator is to add two operands. So a single operator '+', when placed between integer operands, adds them and when placed between string operands, concatenates them.

Runtime polymorphism: This type of polymorphism is achieved by Function Overriding. Function overriding occurs when a derived class has a definition for one of the member functions of the base class. That base function is said to be overridden.

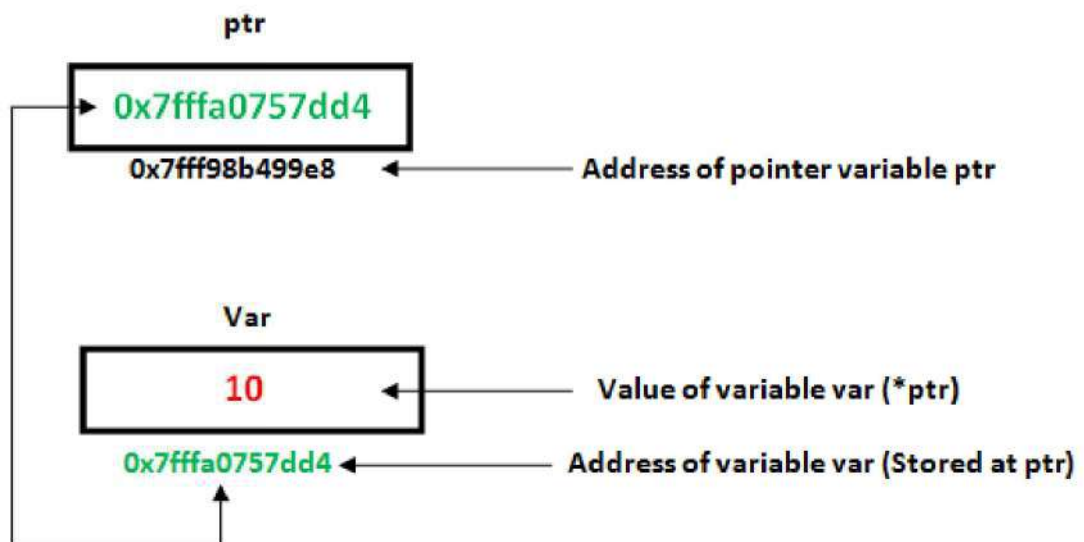
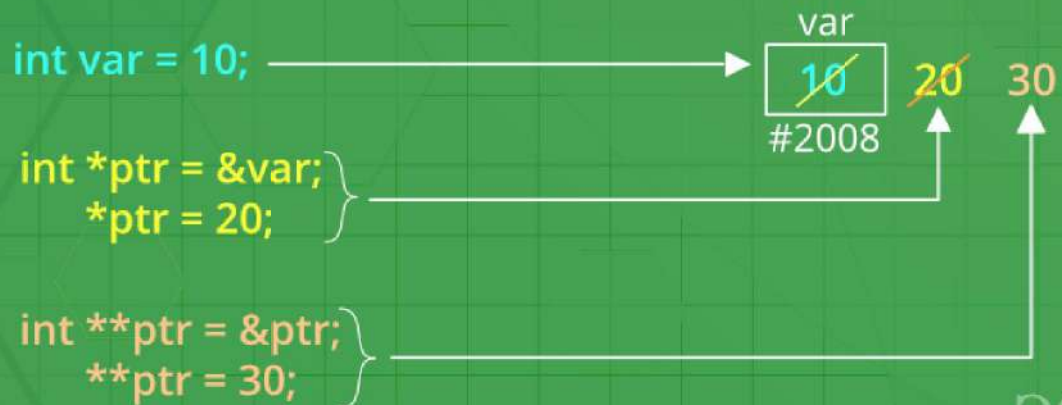
Pointers in C++:

Pointers are symbolic representation of addresses. They enable programs to simulate call-by-reference as well as to create and manipulate dynamic data structures. It's general declaration in C/C++ has the format:

```
datatype *var_name;
```

```
int *ptr;    //ptr can point to an address which holds int data
```

How pointer works in C



```
#include <iostream>
using namespace std;
void geeks()
{
    int var = 10;

    //declare pointer variable
    int *ptr;
```

```

//note that data type of ptr and var must be same
ptr = &var;

// assign the address of a variable to a pointer
cout << "Value at ptr = " << ptr << "\n";
cout << "Value at var = " << var << "\n";
cout << "Value at *ptr = " << *ptr << "\n";
}

//Driver program
int main()
{
    geeks();
}
Value at ptr = 0x7fffa0757dd4
Value at var = 10
Value at *ptr = 10

```

POINTER TO OBJECTS:

A Variable That Holds an Address value is called a Pointer variable or simply pointer. Pointer point to Simple data types likes int, char, float etc. So similar to these type of data type, Objects can also have an address, so there is also a pointer that can point to the address of an Object.

A pointer to a C++ class is done exactly the same way as a pointer to a structure and to access members of a pointer to a class you use the member access operator -> operator, just as you do with pointers to structures. Also as with all pointers, you must initialize the pointer before using it.

Let us try the following example to understand the concept of pointer to a class –

```

#include <iostream>
using namespace std;
class Box {
public:
    // Constructor definition
    Box(double l = 2.0, double b = 2.0, double h = 2.0) {
        cout << "Constructor called." << endl;
        length = l;
        breadth = b;
        height = h;
    }
    double Volume() {
        return length * breadth * height;
    }
private:
    double length;    // Length of a box
    double breadth;   // Breadth of a box
    double height;    // Height of a box
};

int main(void) {
    Box Box1(3.3, 1.2, 1.5);    // Declare box1
    Box Box2(8.5, 6.0, 2.0);    // Declare box2
    Box *ptrBox;                // Declare pointer to a class.

    // Save the address of first object
    ptrBox = &Box1;

    // Now try to access a member using member access operator
    cout << "Volume of Box1: " << ptrBox->Volume() << endl;

    // Save the address of second object
    ptrBox = &Box2;

    // Now try to access a member using member access operator
    cout << "Volume of Box2: " << ptrBox->Volume() << endl;
    return 0;
}

```

Pointer to derived classes:

A Variable That Holds an Address value is called a Pointer variable or simply pointer. Pointer point to Simple data types likes int, char, float etc. So similar to these type of data type, Objects can also have an address, so there is also a pointer that can point to an Object of derived class.

```

//Pointer to derived class
#include <iostream>
using namespace std;
class A
{
public:

```



```

virtual void display()
{
    cout << "Base class is invoked"<<endl;
}
};
class B:public A
{
    public:
    void display()
    {
        cout << "Derived Class is invoked"<<endl;
    }
};
int main()
{
    B * a; //pointer of Derived class
    B b;   //object of derived class
    a = &b;
    a->display(); //Late Binding occurs
}

```

‘this’ pointer in C++:

To understand ‘this’ pointer, it is important to know how objects look at functions and data members of a class.

1. Each object gets its own copy of the data member.
2. All-access the same function definition as present in the code segment.

Meaning each object gets its own copy of data members and all objects share a single copy of member functions.

Then now question is that if only one copy of each member function exists and is used by multiple objects, how are the proper data members are accessed and updated?

The compiler supplies an implicit pointer along with the names of the functions as ‘this’.

Every object in C++ has access to its own address through an important pointer called **this** pointer. The **this** pointer is an implicit parameter to all member functions. Therefore, inside a member function, this may be used to refer to the invoking object.

Following are the situations where ‘this’ pointer is used:

1) When local variable’s name is same as member’s name

```

#include<iostream>
using namespace std;

/* local variable is same as a member's name */
class Test
{
    private:
        int x;
    public:
        void setX (int x)
        {
            // The 'this' pointer is used to retrieve the object's x
            // hidden by the local variable 'x'

```

```

        this->x = x;
    }
    void print() { cout << "x = " << x << endl; }
};

```

```

int main()
{
    Test obj;
    int x = 20;
    obj.setX(x);
    obj.print();
    return 0;
}

```

2) To return reference to the calling object

```

/* Reference to the calling object can be returned */
Test& Test::func ()
{
    // Some processing
    return *this;
}

```

When a reference to a local object is returned, the returned reference can be used to **chain function calls** on a single object.

```

#include<iostream>
using namespace std;
class Test
{
private:
    int x;
    int y;
public:
    Test(int x = 0, int y = 0) { this->x = x; this->y = y; }
    Test &setX(int a) { x = a; return *this; }
    Test &setY(int b) { y = b; return *this; }
    void print() { cout << "x = " << x << " y = " << y << endl; }
};

int main()
{
    Test obj1(5, 5);
    // Chained function calls. All calls modify the same object
    // as the same object is returned by reference
    obj1.setX(10).setY(20);
    obj1.print();
    return 0;
}

```

C++ virtual function

- A C++ virtual function is a member function in the base class that you redefine in a derived class. It is declared using the virtual keyword.
- It is used to tell the compiler to perform dynamic linkage or late binding on the function.
- A 'virtual' is a keyword preceding the normal declaration of a function.
- When the function is made virtual, C++ determines which function is to be invoked at the runtime based on the type of the object pointed by the base class pointer.

```

//without virtual functions
#include <iostream>
using namespace std;
class A
{
    int x=5;
    public:
    void display()
    {
        cout << "Value of x is : " << x<< endl;
    }
};
class B: public A
{
    int y = 10;
    public:
    void display()
    {
        cout << "Value of y is : " <<y<< endl;
    }
};
int main()
{
    A *a;
    B b;
    a = &b;
    a->display();
    return 0;
}

```

Note:- The Arrow (->) operator is used to access the member of a class using pointer to the object.

```

//virtual function
#include <iostream>
Using namespace std;
class A {
{
    public:
    virtual void display()
    {
        cout << "Base class is invoked" <<endl;
    }
}

```

```

};
class B:public A
{
    public:
    void display()
    {
        cout << "Derived Class is invoked"<<endl;
    }
};
int main()
{
    A* a;    //pointer of base class
    B b;     //object of derived class
    a = &b;
    a->display(); //Late Binding occurs
}

```

Pure Virtual Function

- A virtual function is not used for performing any task. It only serves as a placeholder.
- When the function has no definition, such function is known as "**do-nothing**" function.
- The "**do-nothing**" function is known as a **pure virtual function**. A pure virtual function is a function declared in the base class that has no definition relative to the base class.
- A class containing the pure virtual function cannot be used to declare the objects of its own, such classes are known as abstract base classes.
- The main objective of the base class is to provide the traits to the derived classes and to create the base pointer used for achieving the runtime polymorphism.

Pure virtual function can be defined as:

```

virtual void display() = 0;
//Program to demonstrate pure virtual function
#include <iostream>
using namespace std;
class Base
{
    public:
    virtual void show() = 0;
};
class Derived : public Base
{
    public:
    void show()

```

```
{
    cout << "Derived class is derived from the base class." << std::endl;
}
};
int main()
{
    Base *bptr;
    //Base b;
    Derived d;
    bptr = &d;
    bptr->show();
    return 0;
}
```

Server Virtualisation: Server virtualization is a process of partitioning the resources of a single server into multiple virtual servers. These virtual servers can run as separate machines. Server virtualization allows businesses to run multiple independent tasks with different configurations using a single (host) server. The process also saves the hardware cost involved in keeping a host of physical servers.

Block and File level Storage Virtualisation: Storage virtualization is a series of servers that are managed by a virtual storage system. The servers aren't aware of exactly where their data is stored. This technology manages the storage of data from multiple users and utilized as a single storage system. storage virtualization software maintains smooth operations and consistent performance despite changes, break down and differences in the connected equipment.

Data virtualization: This is a kind of virtualization in which the data is collected from various sources and managed that in a single server without knowing more about the technical information like how data is collected, stored & formatted. Then the stored data can be arranged in such a way so that its virtual view can be accessed by interested users by using the various cloud services remotely. Many big giant companies are providing their services like Oracle, IBM, At scale, Cdata etc.

Desktop as a service (DaaS)

Desktop as a service (DaaS) is a cloud-based desktop virtualization service hosted by a third party enterprise. The third party cloud provider manages all back-end resources, such as desktop storage, compute, and networking, including the virtual cloud machines that run the desktop operating systems. The provider streams the virtual desktops to end-user devices, allowing anytime, anywhere access to desktops and applications. There are two kinds of desktops available in DaaS. These are:

1. **Persistent desktop:** Users have the ability to customize and save the desktop so that it will look the same way each time when a particular user logs in. Persistent desktop requires more storage than non-persistent desktop.
2. **Non-persistent desktop:** Desktops are wiped each time the user logs out—they are merely a way to access shared cloud services.

DaaS Advantages:

- Easy platform migration
- Total cost reduction
- Minimized complexity
- Disaster recovery
- Uninterrupted connectivity
- Increased performance
- Personalization
- Reliability
- Data security